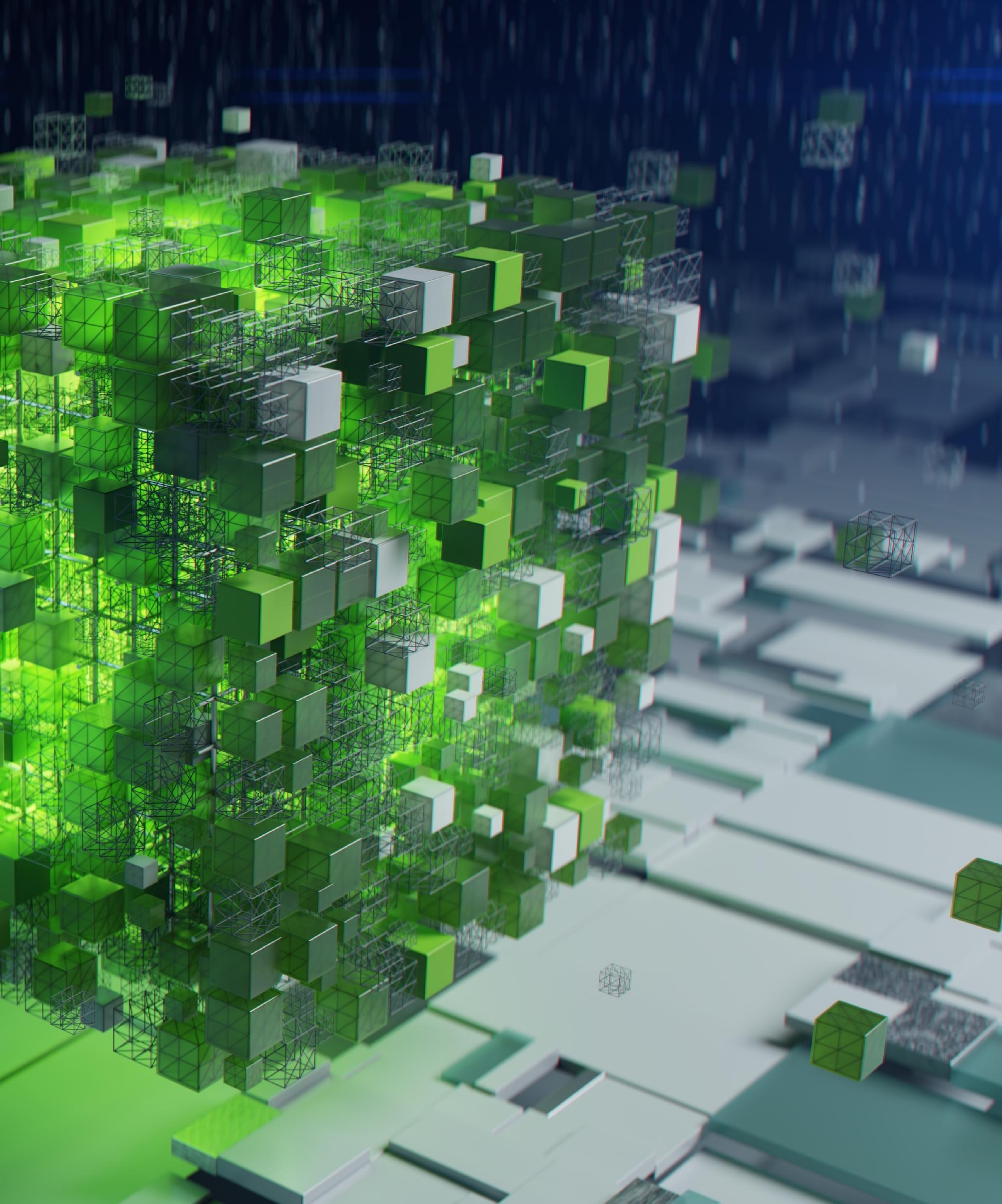




Python on GPUs

Andy R. Terrel, PhD
CUDA Product Group Lead (Languages and Libraries)
Senior Manager, Technical Product Management



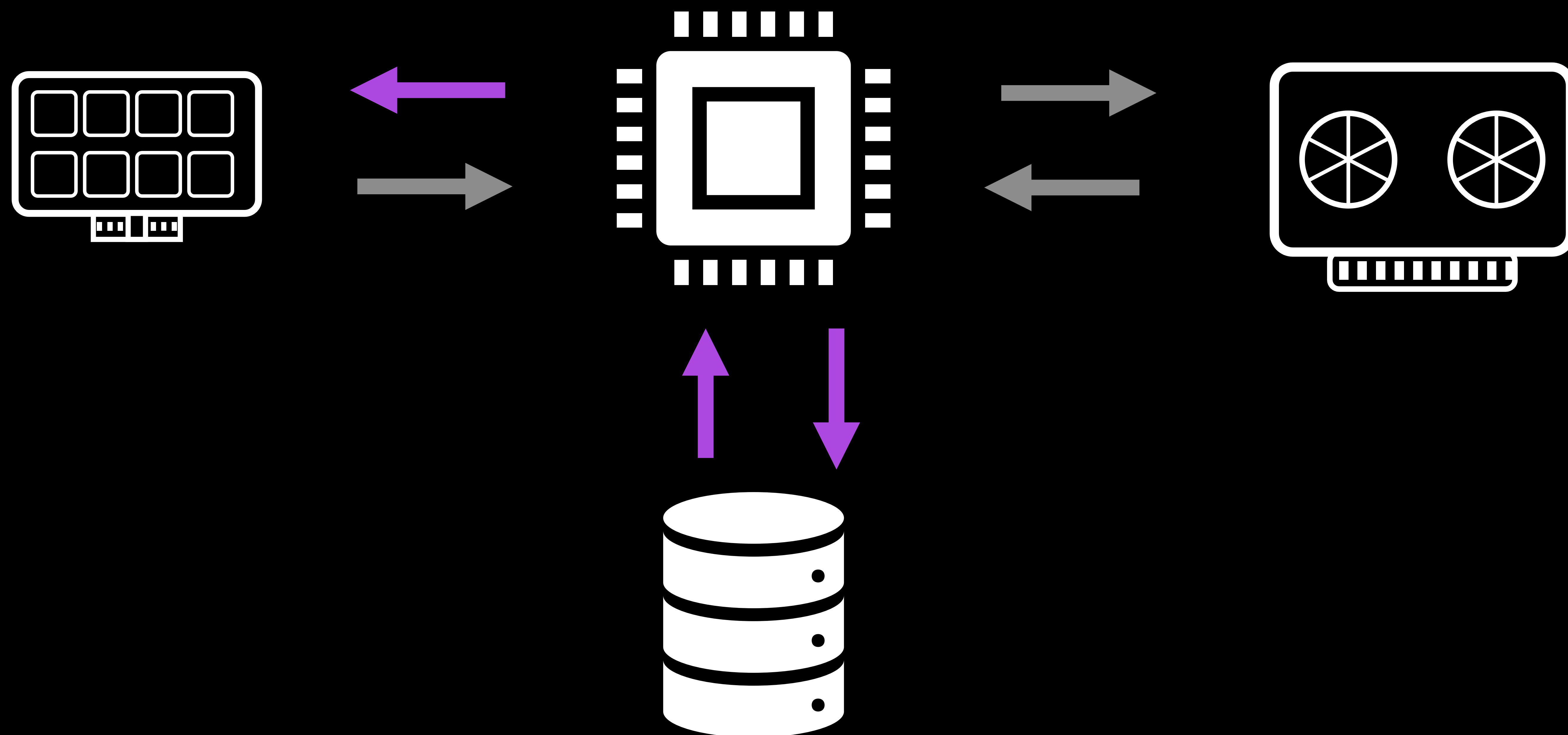
Agenda

Exposing the GPU

The Evolution of Hardware

Tiling languages

Computing on a GPU



Single-GPU accelerated libraries.



Exposures of the CUDA Ecosystem

Array libraries provide high level interface to GPU programming

```
import numpy as np
import cupy as cp
```

```
a = 5.0
```

```
x_cpu = np.random.rand(n)
```

```
y_cpu = np.random.rand(n)
```

```
x_gpu = cp.asarray(x_cpu)
```

```
y_gpu = cp.asarray(y_cpu)
```

```
result_gpu = a*x_gpu + y_gpu
```

```
# move the array to the host.
```

```
result_cpu = result_gpu.get()
```



Exposures of the CUDA Ecosystem

Data Science libraries often focus on different types

```
import cudf
```

```
df = cudf.DataFrame(  
    {'angles': [0, 3, 4], 'degrees': [360, 180, 360]},  
    index=['circle', 'triangle', 'rectangle']  
)
```

```
df.multiply(1)
```

```
#      angles  degrees  
# circle      0    360  
# triangle    3    180  
# rectangle   4    360
```



RAPIDS

Exposures of the CUDA Ecosystem

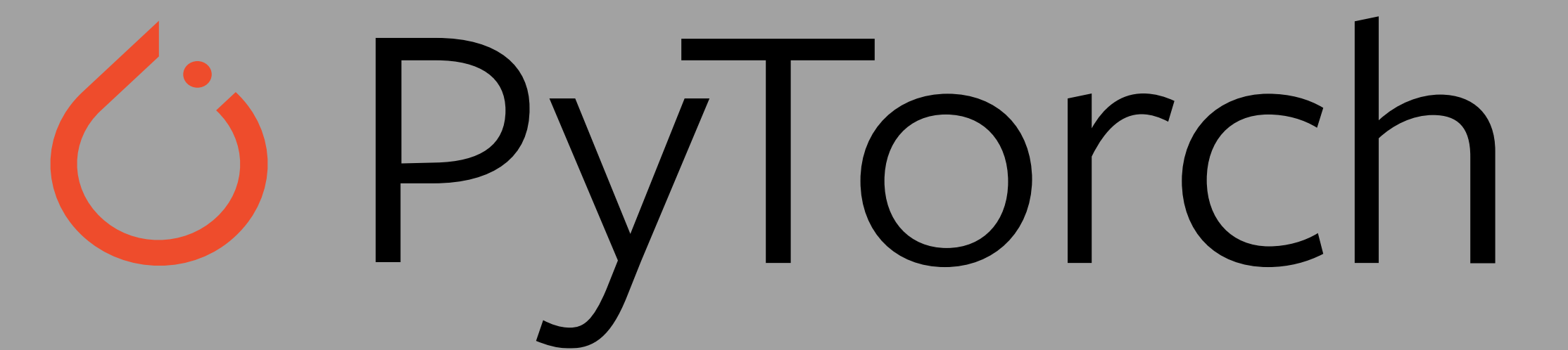
Machine Learning Libraries that hide the details of the GPU vs CPU

```
import torch

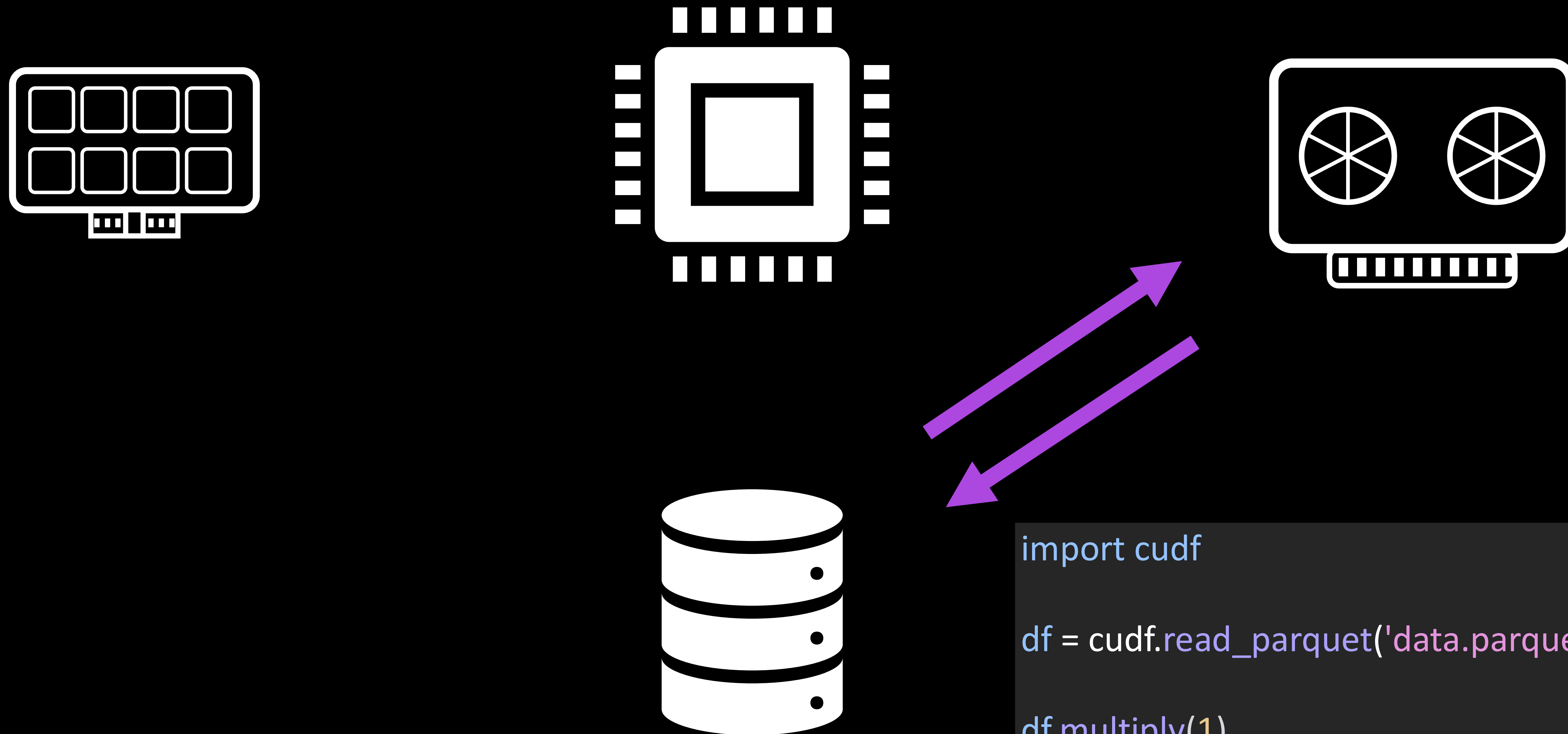
def saxpy(a: float, x: torch.Tensor, y: torch.Tensor)
    -> torch.Tensor:
    return a * x + y

# Example usage:
a = 2.0
x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([4.0, 5.0, 6.0])

result = saxpy(a, x, y)
print(result) # Output: tensor([6., 9., 12.] )
```



Computing on a GPU



With an evolved single-GPU ecosystem, we now have memory compatible CUDA libraries.



CUDA Python 1.0

Accelerated Python requires more than just Python CUDA kernels

Kernel Authoring
CUDA kernels in Python

Native Python GPU compilers
(Thread, Tile & Tensor Core)

numba-cuda

cuTile

CUTLASS DSL

CUDA Libraries
Pythonic interfaces
Host & device libraries
Communications APIs

CUDA Core Compute Libraries

cuda.compute

Host-callable
parallel algorithms

cuda.coop

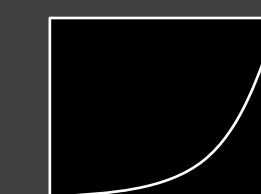
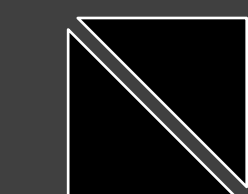
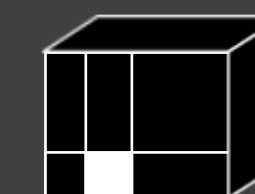
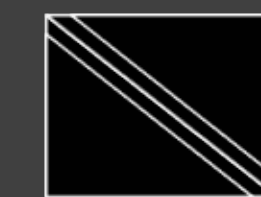
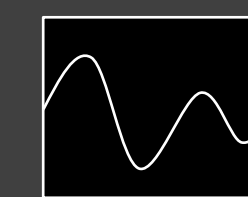
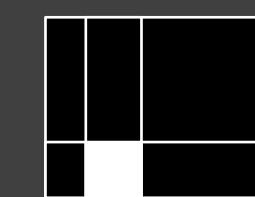
Device primitives
for collective
operations



nvmath-python

Host APIs

Device APIs



Low-level bindings

Communication Libraries
(host & device interfaces)

nccl4py

Python bindings for NCCL

nvshmem4py

Python bindings for NVSHMEM

Runtime System
Device management
Memory allocation
Streams & Synchronization
CUDA Graphs
JIT compilation

cuda.core

High-level **pythonic** object model for CUDA

cuda.bindings

Low-level 1:1 C → Python bindings

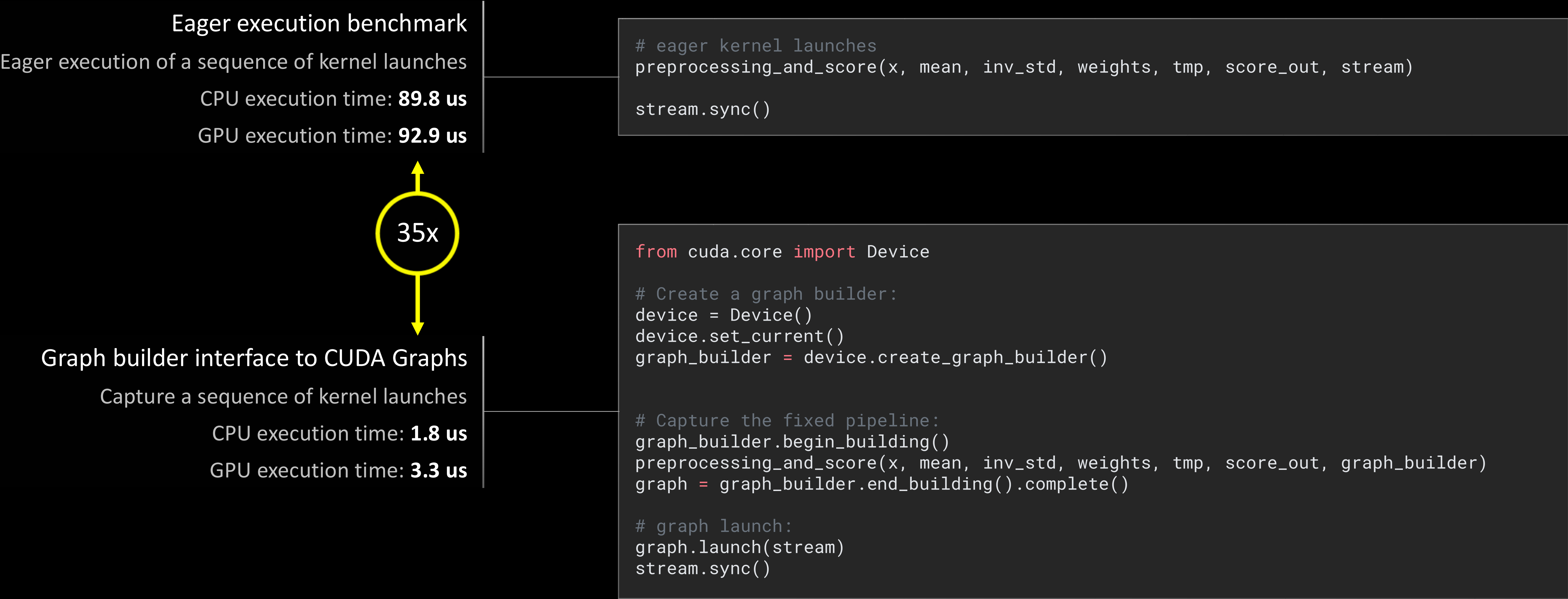
CUDA C++ Runtime

Run-Time Compilation

Linker / LTO

Peak Performance in Python: CUDA Core Graph Builder

Minimise launch and execution overheads



Peak Performance in Python: JIT Fusion

Fuse operations with `cuda.compute` iterators & `nvmath-python` prologs & epilogs

PyTorch with `torch.compile`
Generates two kernels: `exp()` and `sum()`
Execution time: **39.7 us**

4.8x

`cuda.compute` with iterators
Single kernel: fused reduction
Execution time: **8.2 us**

```
import torch

size = 1024 * 1024
x = torch.randn(size, device='cuda')
y = torch.randn_like(x)
d_out = torch.empty(1)

f = torch.compile(lambda x, y: torch.exp(x * y).sum())
d_out = f(x, y)
```

```
import math
import cuda.compute
from cuda.compute import (
    OpKind, ZipIterator, TransformIterator, Determinism
)
import torch

size = 1024 * 1024
x = torch.randn(size, device='cuda')
y = torch.randn_like(x)
d_out = torch.empty(1)

def exp(t):
    return math.exp(t[0] * t[1])

input_it = ZipIterator(x, y)
transform_it = TransformIterator(input_it, exp)

cuda.compute.reduce_into(
    transform_it, d_out, OpKind.PLUS, size, h_init,
    determinism=Determinism.NOT_GURANTEED
)
```


Kernel Fusion is Key

JIT dynamic-typed GEMM with epilogues: $D = \text{epilog}(\alpha A \bullet B + \beta C)$

CuPy (Unfused kernels)

```
import cupy as cp

m, n, k = 10_000_000, 40, 10

a = cp.random.rand(m, k, dtype=cp.float32)
b = cp.random.rand(k, n, dtype=cp.float32)
c = cp.random.rand(m, n, dtype=cp.float32)

alpha = 1.5
beta = 0.5
d = alpha * cp.matmul(a, b) + beta * c
```

Unfused kernels execute each operator sequentially

1 2 3 4

nvmath-python (fused kernels)

```
import nvmath
import cupy as cp

m, n, k = 10_000_000, 40, 10

a = cp.random.rand(m, k, dtype=cp.float32)
b = cp.random.rand(k, n, dtype=cp.float32)
c = cp.random.rand(m, n, dtype=cp.float32)

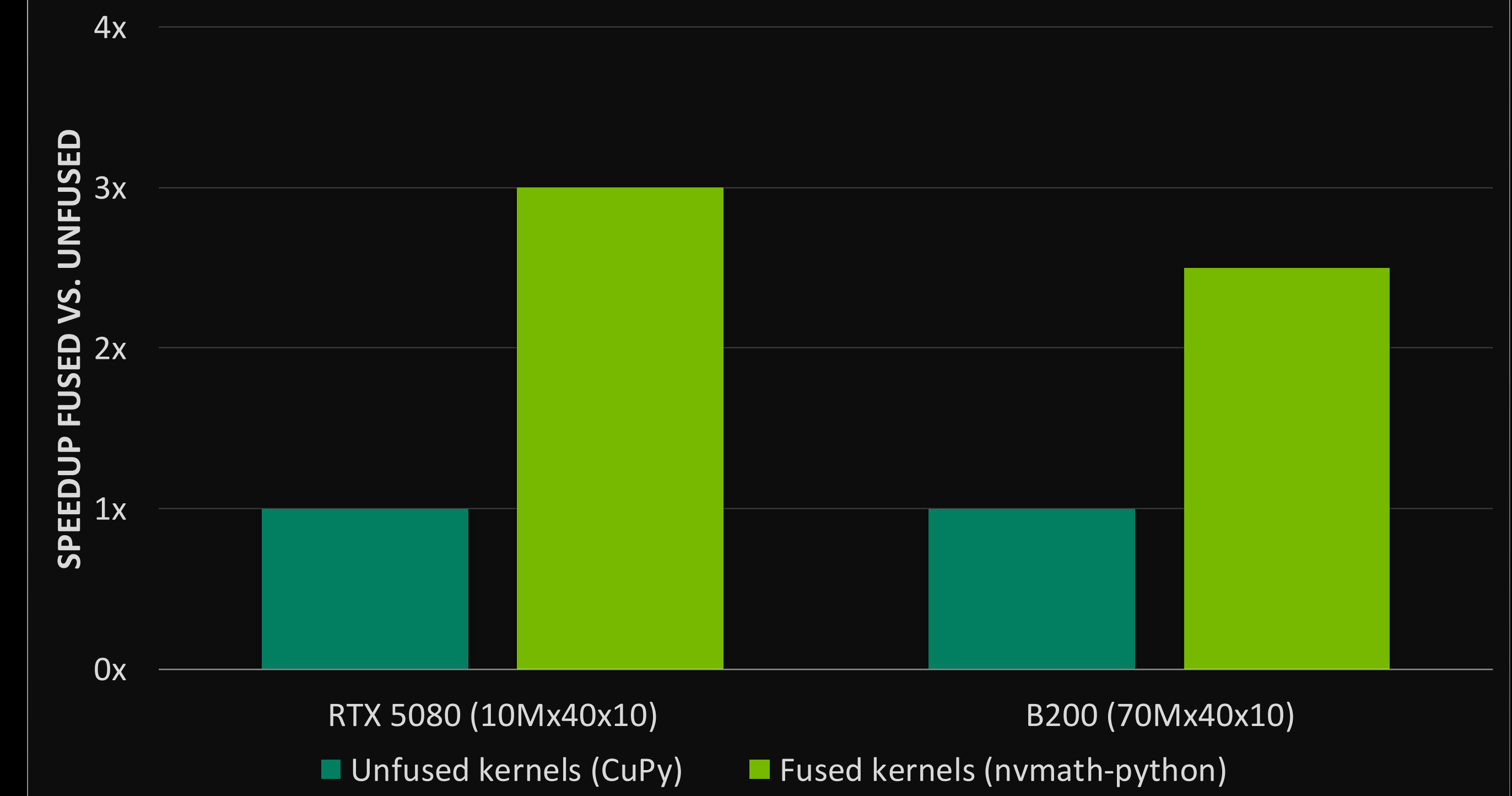
alpha = 1.5
beta = 0.5
d = nvmath.linalg.advanced.matmul(
    a, b, c, alpha=alpha, beta=beta
)
```

Host-side JIT combines all operations into a single kernel

1

Using nvmath-python to fuse GEMM+epilog

Tall-and-narrow matrices



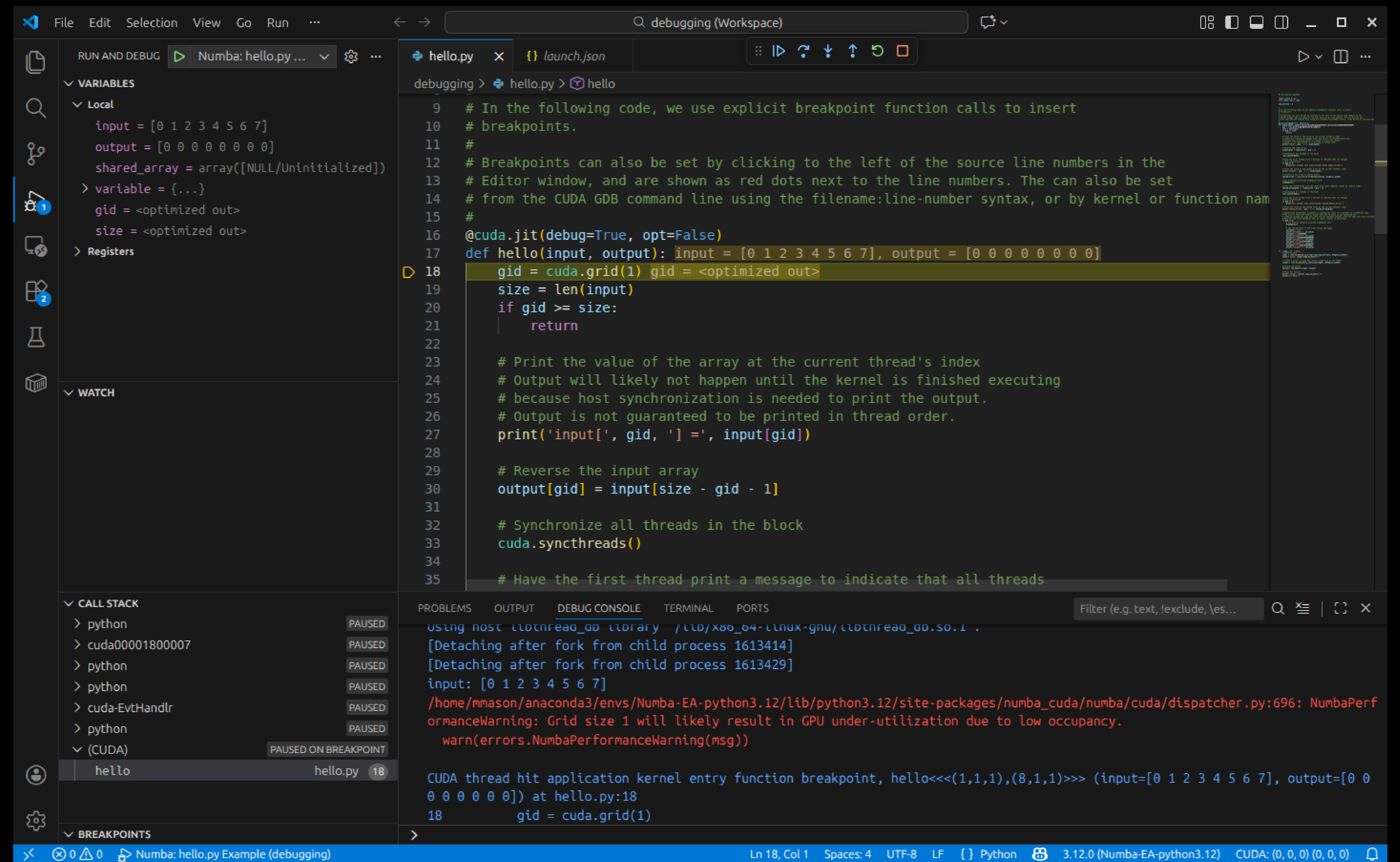
Finally: Debug Numba CUDA Kernels

Numba CUDA provides a CUDA target for the Numba Python JIT Compiler

Native debugging of Python device code

- Breakpoints
- Single-stepping code
- Variable inspection
- etc.

Enabled via cuda-gdb and
Nsight Visual Studio Code Edition



<https://nvidia.github.io/numba-cuda/user/debugging.html>

Introducing Nsight Python

Profiling directly from Python

Nsight Python Beta now live

Python API for kernel profiling across multiple configurations

Automatic plotting for visualization and comparisons

Access profiling data directly from Python objects

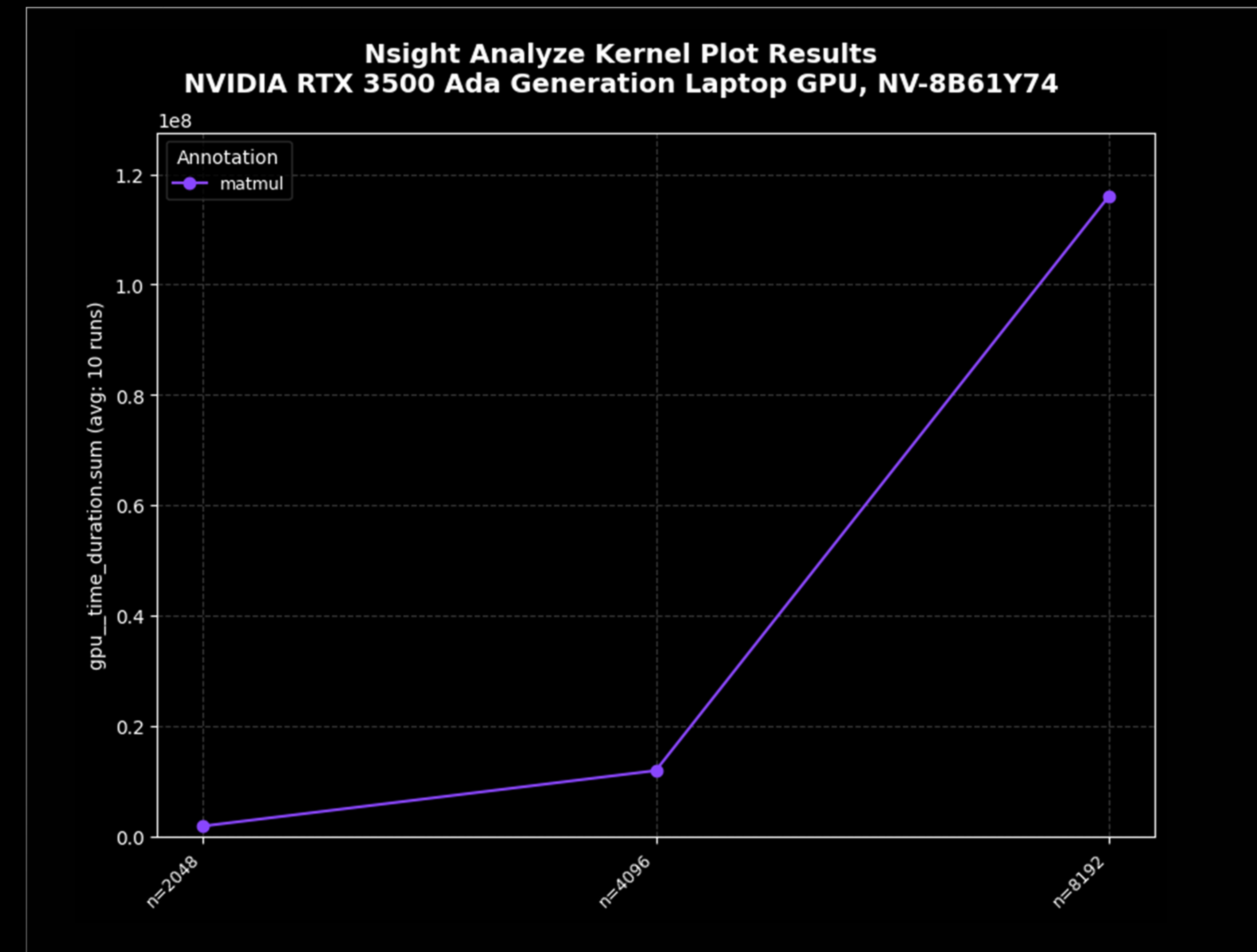
Available on PyPi and GitHub

<https://pypi.org/project/nsight-python/>

<https://github.com/NVIDIA/nsight-python>

```
@nsight.analyze.plot("02_parameter_sweep.png")
@nsight.analyze.kernel(configs=sizes, runs=10)
def benchmark_matmul_sizes(n: int) -> None:
    """
    Benchmark matrix multiplication across different sizes.
    The 'n' parameter comes from the configs list.
    """
    a = torch.randn(n, n, device="cuda")
    b = torch.randn(n, n, device="cuda")

    with nsight.annotate("matmul"):
        _ = a @ b
```



<https://developer.nvidia.com/nsight-python>

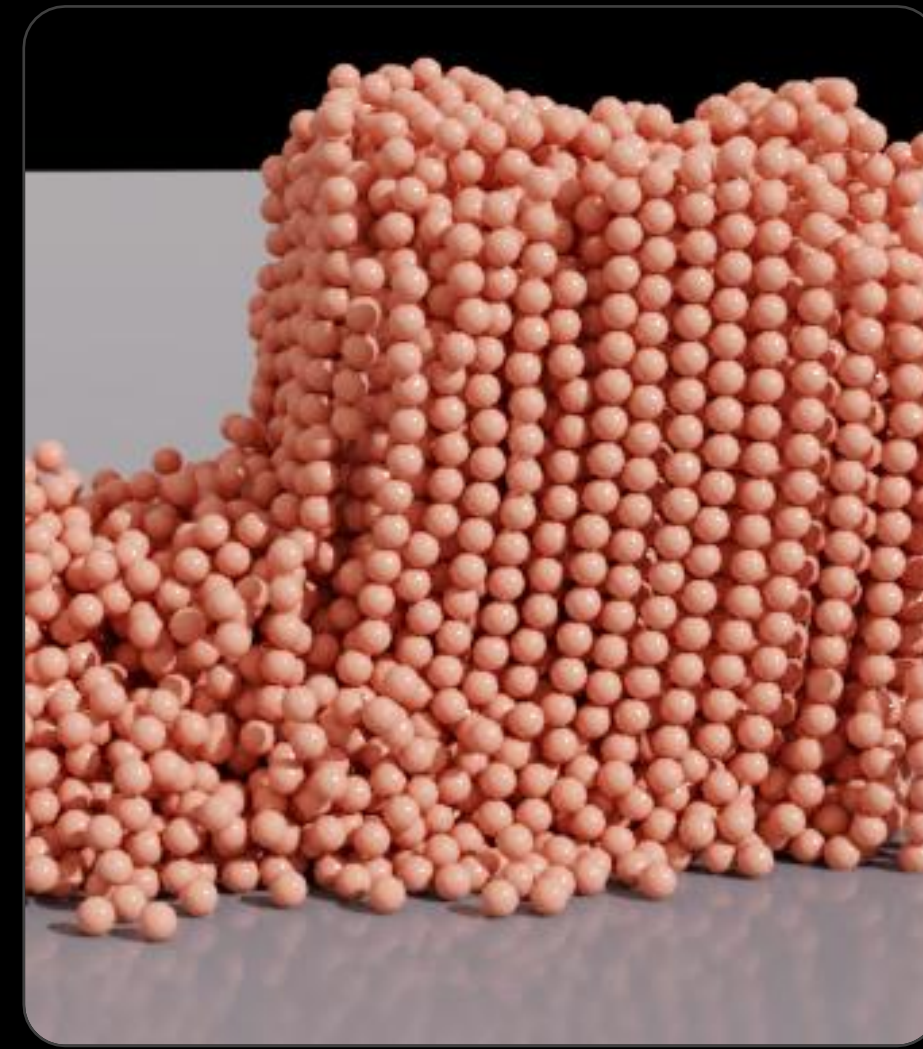
CUDA-X Accelerates Every Industry



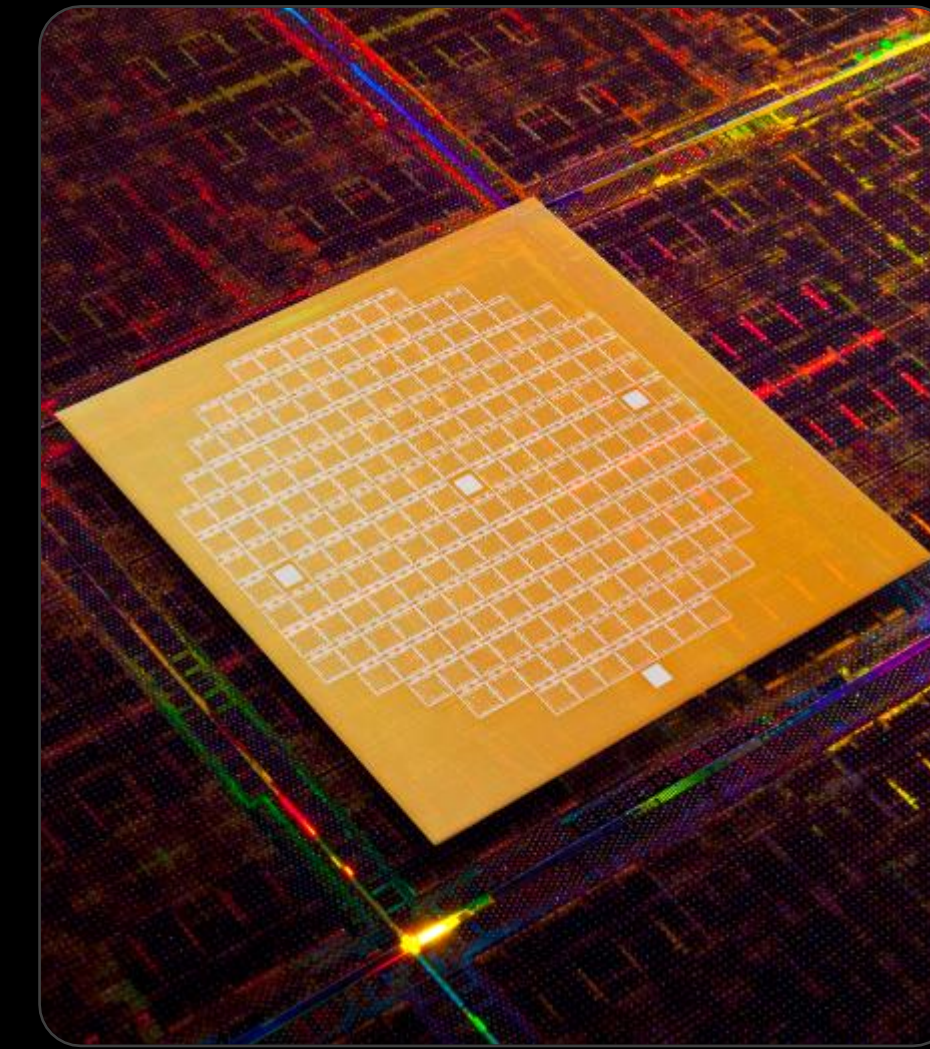
cuDSS
CAE



PhysicsNeMo
AI Physics



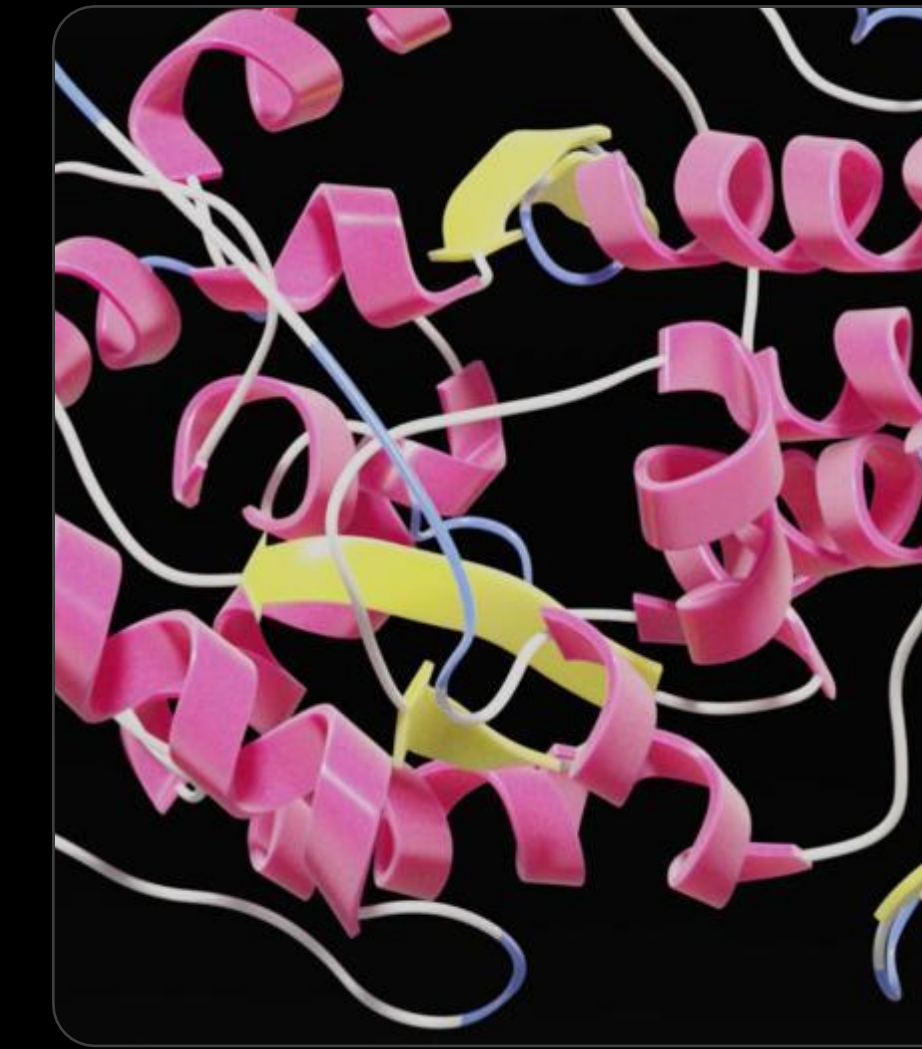
Warp
Physical Simulation



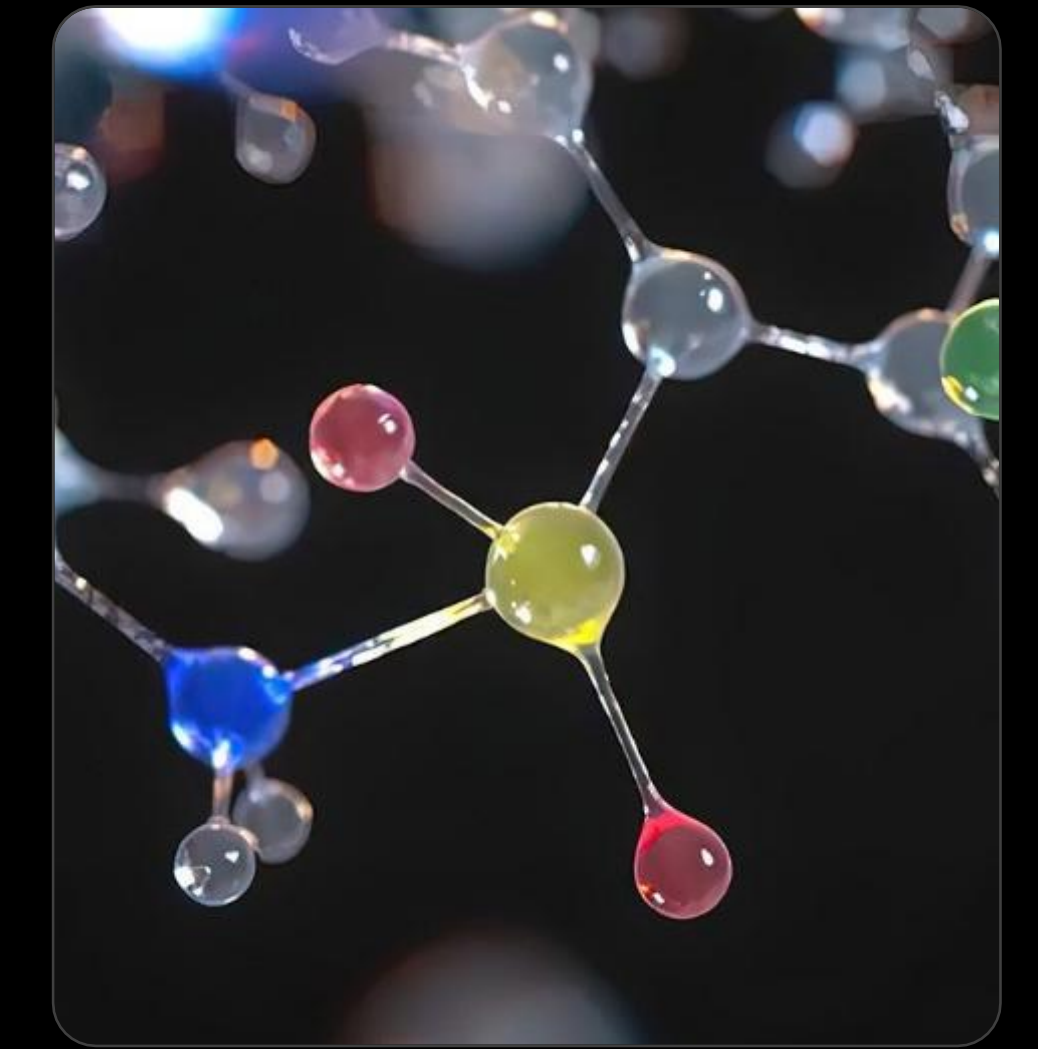
cuLitho
Computational Lithography



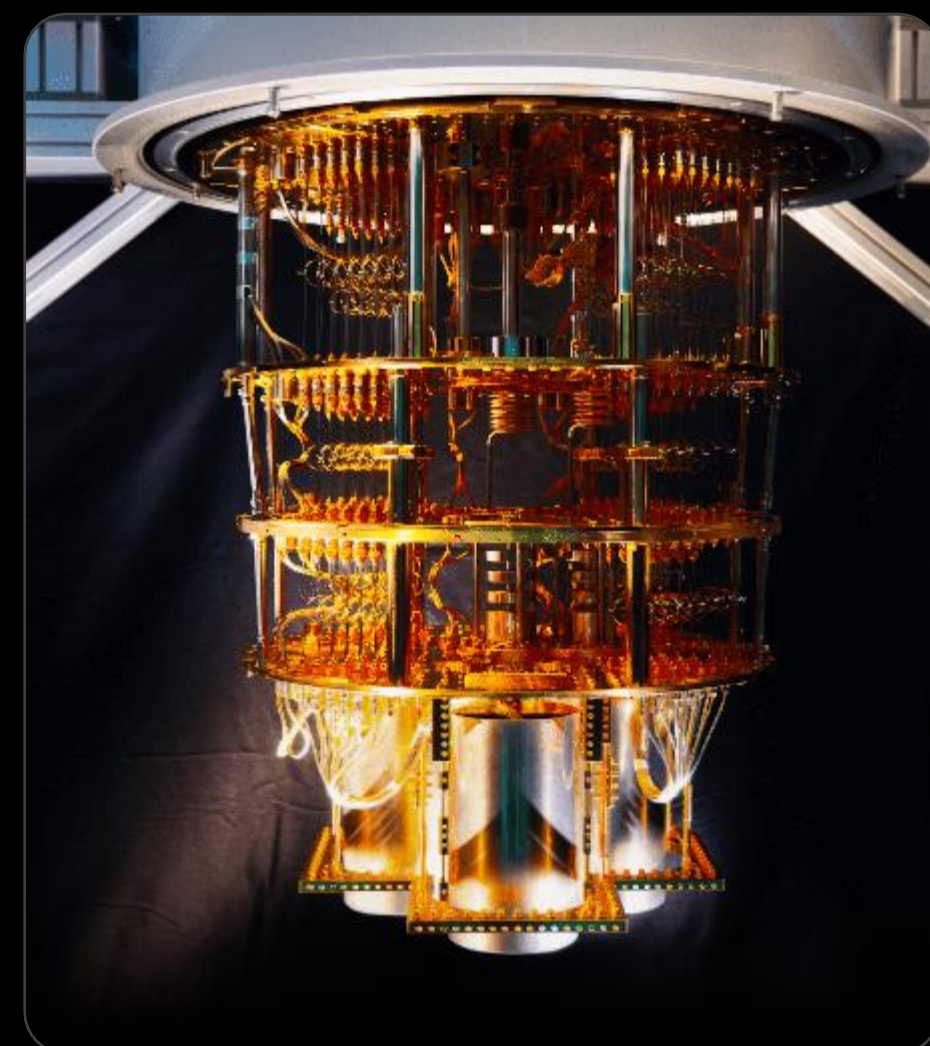
ALCHEMI
AI Materials Science



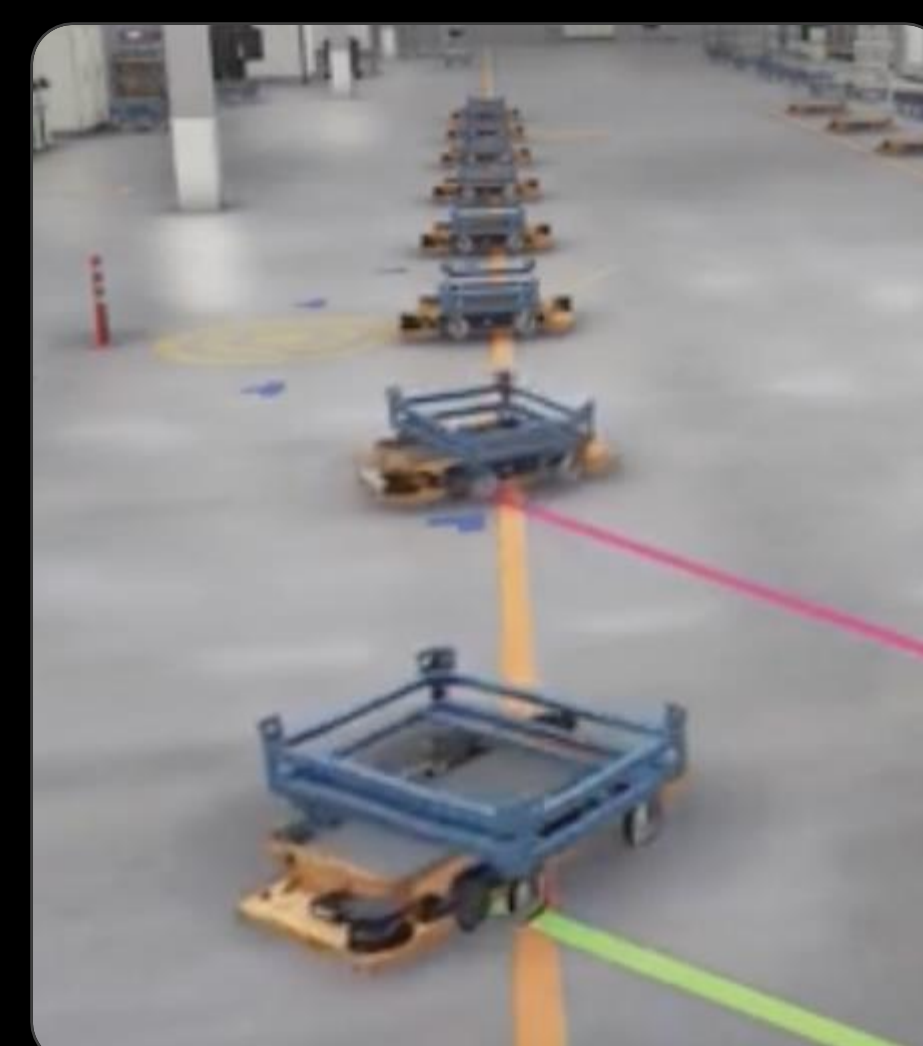
cuEquivariance
Drug & Materials
Discovery



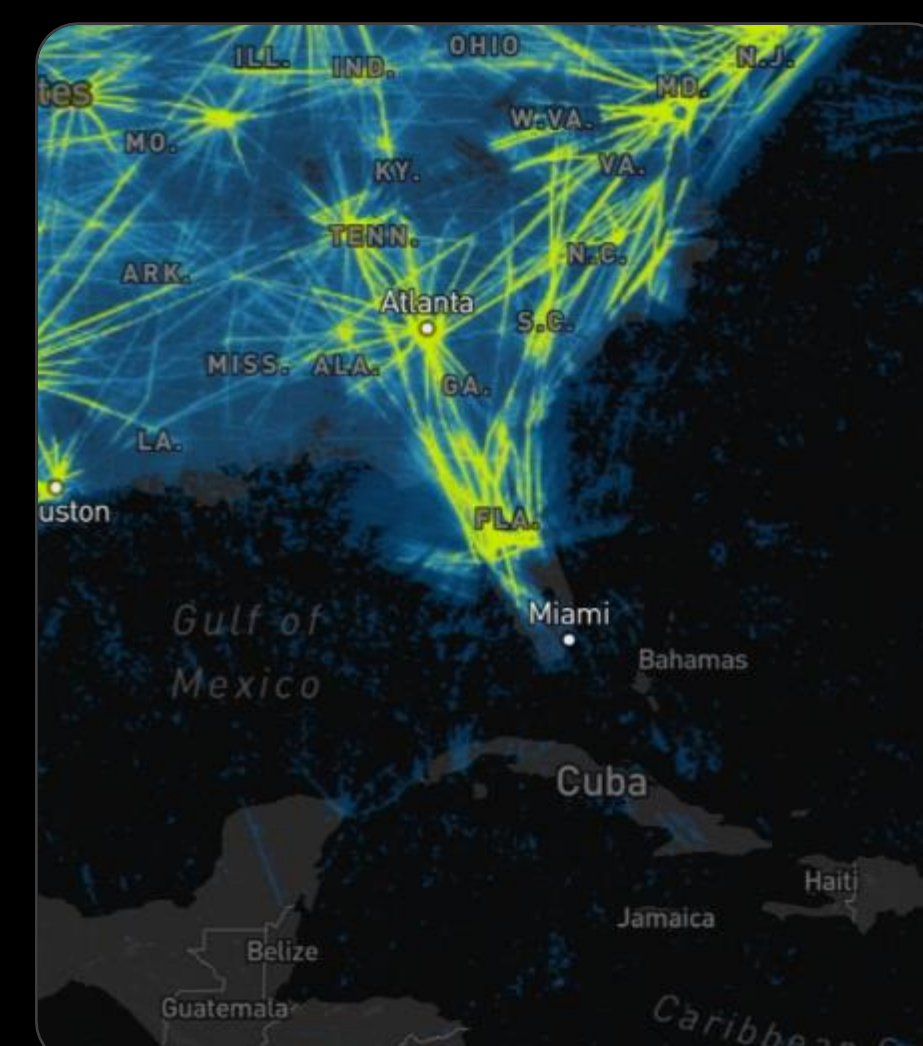
Parabricks
Gene Sequencing



CUDA-Q
Quantum Computing



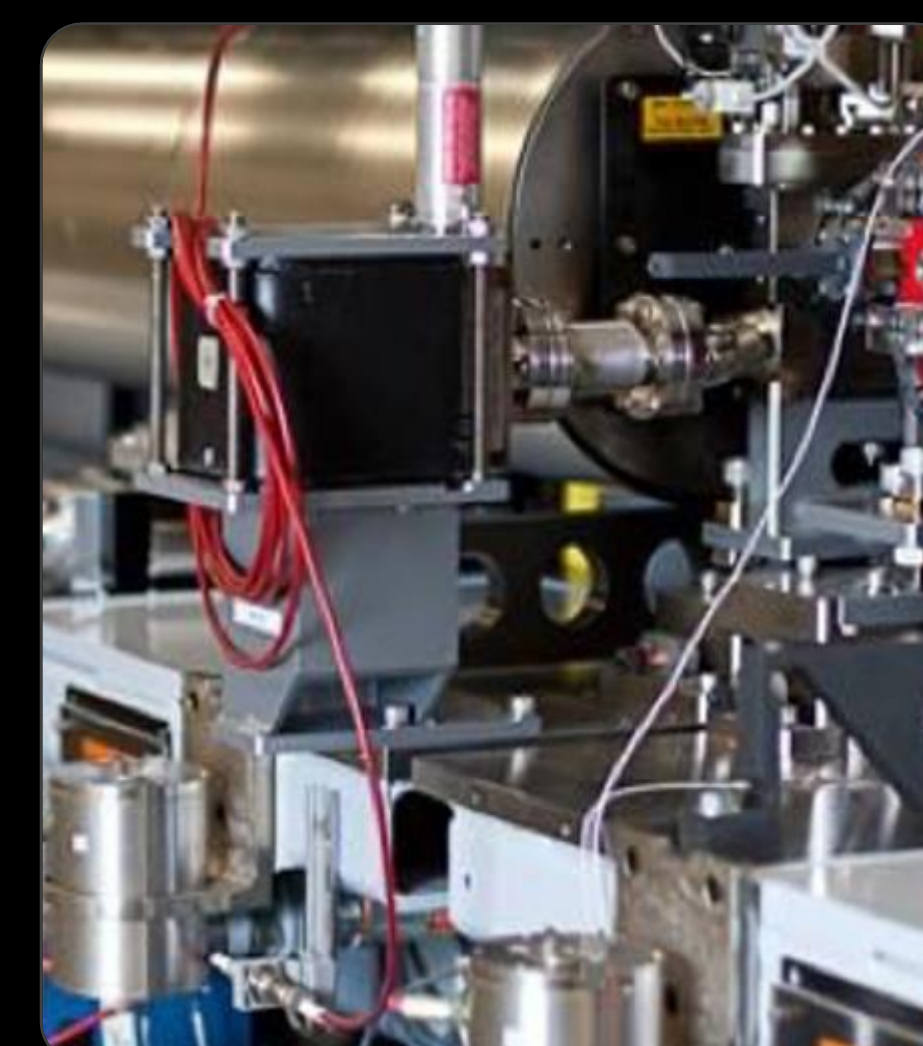
cuOpt
Decision Optimization



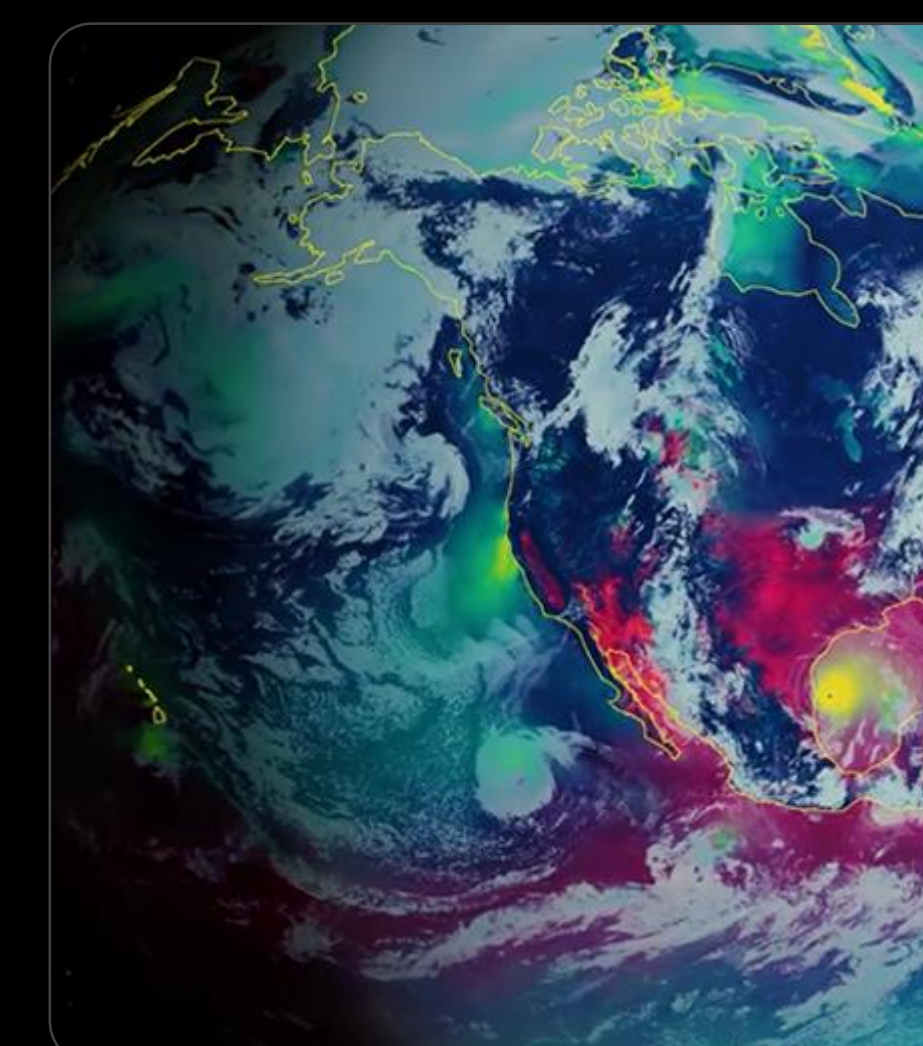
cuDF
Data Processing



cuPyNumeric
Numerical Computing



Holoscan
Edge HPC



Earth-2
Weather Analytics

PHROSTY team

Astrophysicists turning to GPU Developers

Researchers at Duke University, Carnegie Mellon University, University of Pittsburgh, and Lawrence Berkeley National Lab are creating a pipeline to isolate supernovae within survey images taken from the Roman Space Telescope



Roman Space Telescope



Lauren Aldoroty, PhD.

Postdoctoral Researcher
at Duke University



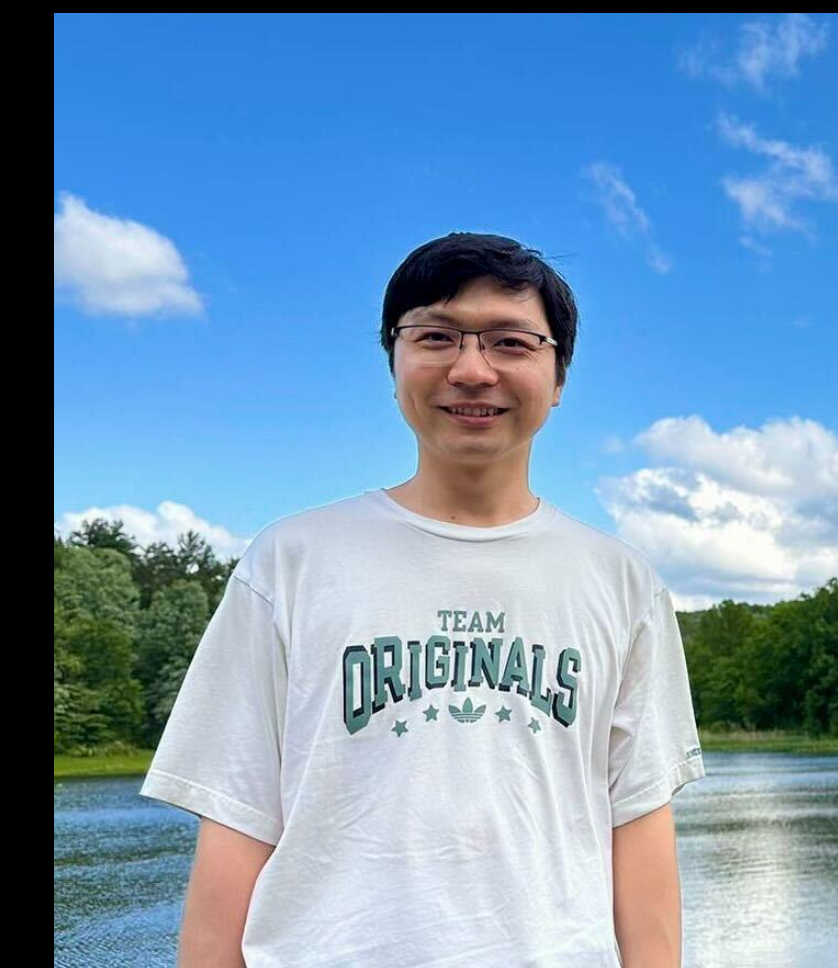
Robert Knop, PhD

Scientific
computational
astrophysicist at LBNL



Michael Wood-Vasey, PhD

Professor of Astronomy,
University of Pittsburgh



Lei Hu, PhD.

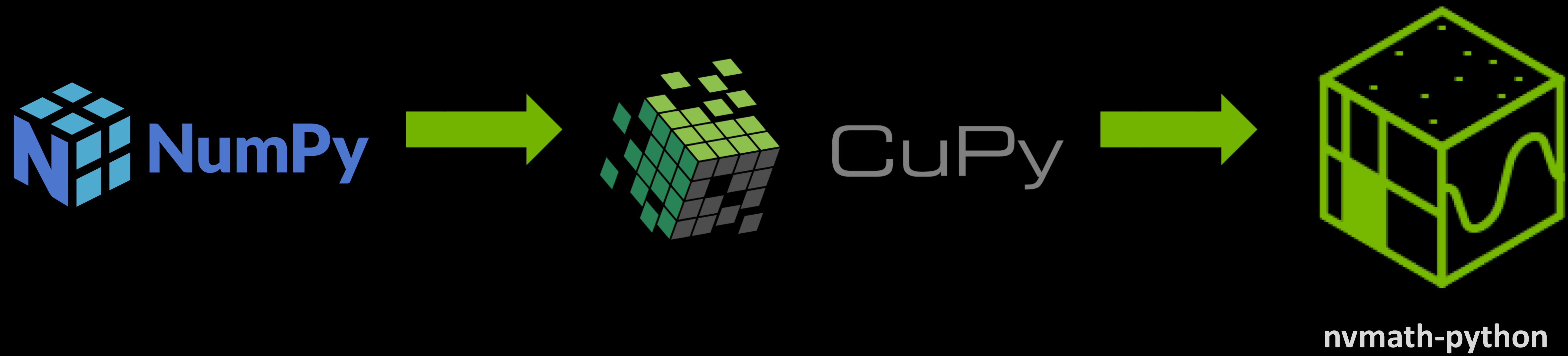
Postdoc at Carnegie
Mellon University working
on Time-domain
Astronomy



Shu Liu, PhD.

Graduate Student,
Astronomy,
University of Pittsburgh

To an optimized GPU Library



nvmath-python: Accelerated Math Libraries For Accelerated Python

A Pythonic interface to high-performance GPU libraries

Reimagining Python Math Libraries

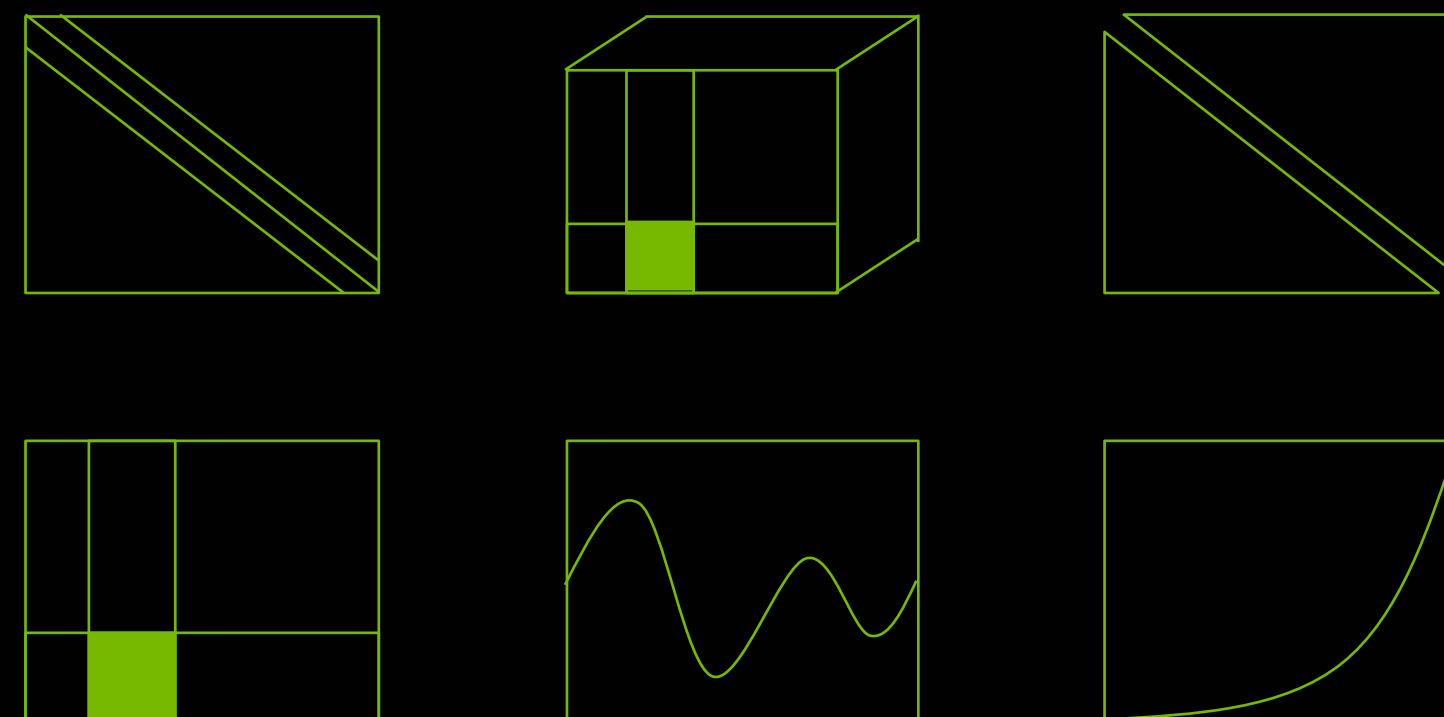
Pythonic APIs for peak performance

Just-in-time compilation for kernel fusion

Scalability without major code rewrite

Open-source, interoperates with existing ecosystem

Available via Conda, Pip & GitHub



Stateful APIs allow amortizing planning/autotuning cost through multiple executions

Stateless APIs are for greatest convenience

Interoperates with popular tensor libraries and frameworks

Provides low-level Python bindings along with high-level Python APIs

Stateless API

```
import cupy as cp
import nvmath

axes = (0, 1)
a = cp.ones((512, 512, 512), dtype=cp.complex64)

# Forward FFT
b = nvmath.fft.fft(a, axes=axes)

# In-place update in the frequency domain
a[:] = b

# Inverse FFT
c = nvmath.fft.ifft(a, axes=axes)

# Synchronize the default stream
cp.cuda.get_current_stream().synchronize()
```

Double cost for FFT planning

Stateful API

```
import cupy as cp
import nvmath

FWD = nvmath.fft.FFTDirection.FORWARD
INV = nvmath.fft.FFTDirection.INVERSE
a = cp.ones((512, 512, 512), dtype=cp.complex64)

# Create a stateful FFT object 'f'
with nvmath.fft.FFT(a, axes = (0, 1)) as f:
    # Plan the FFT
    f.plan()

    # Execute the FFT
    b = f.execute(direction=FWD)

    # In-place update in the frequency domain
    a[:] = b

    # Execute the new FFT
    c = f.execute(direction=INV)

    # Synchronize the default stream
    cp.cuda.get_current_stream().synchronize()
```

Single FFT planning cost amortized through multiple FFT executions

Nancy Grace Roman Space Telescope

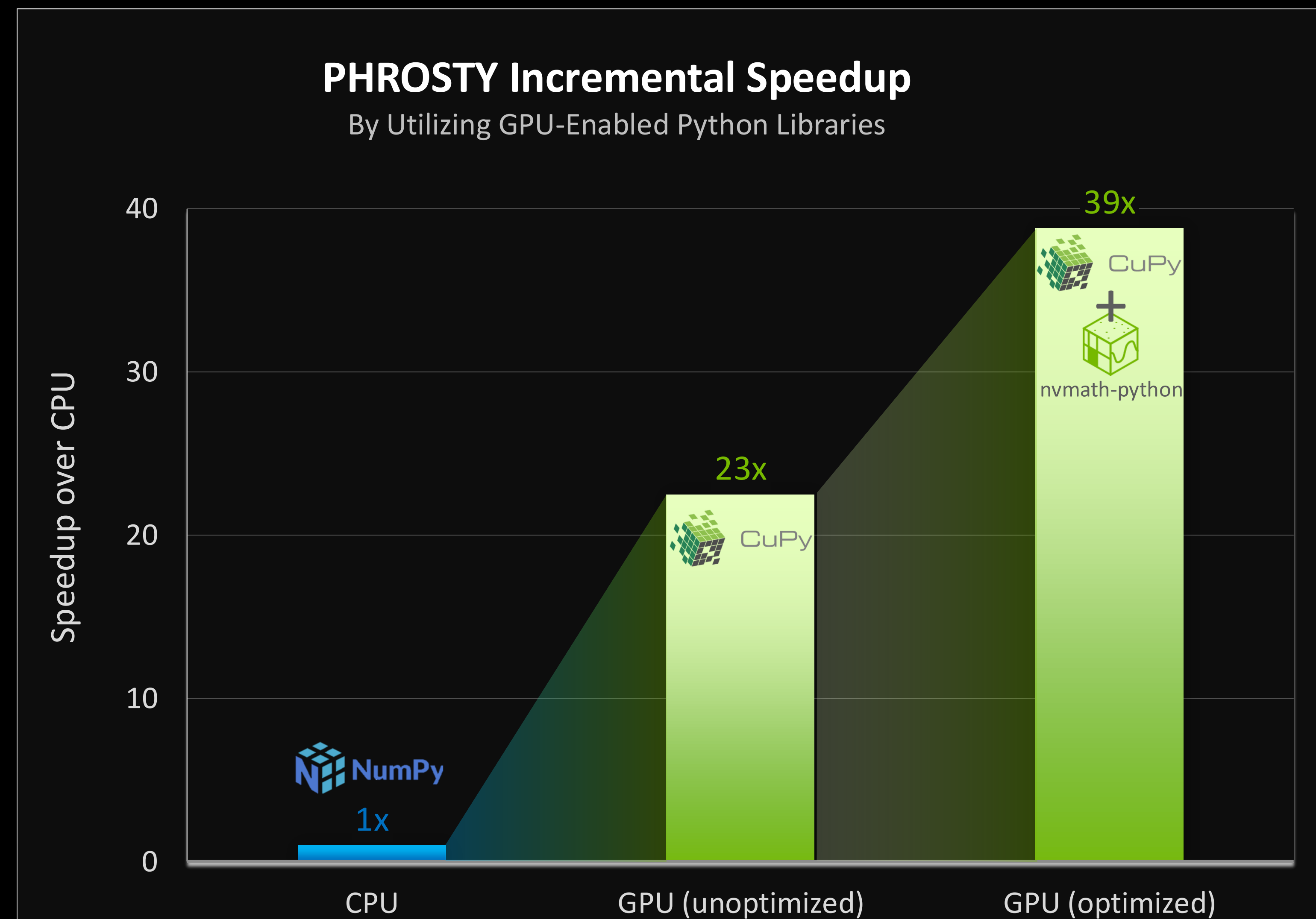
PHROSTY discovery & photometry pipeline for supernova detection

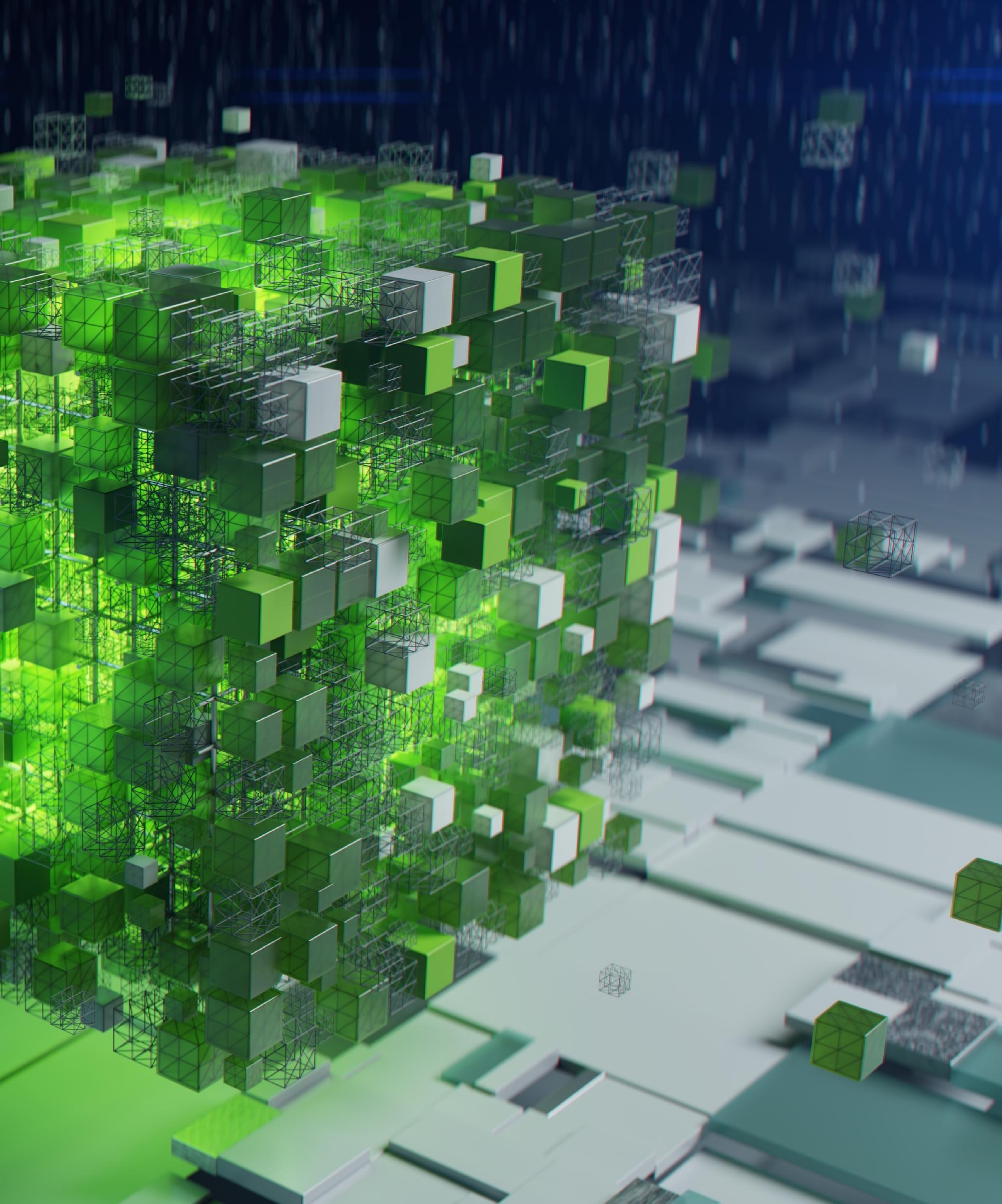
The application consists of 3 parts

Preprocessing involves calculating the point-spread function

sFFT transforms the images into Fourier space and performs differencing between them

Postprocessing involves finding and applying the decorrelation function

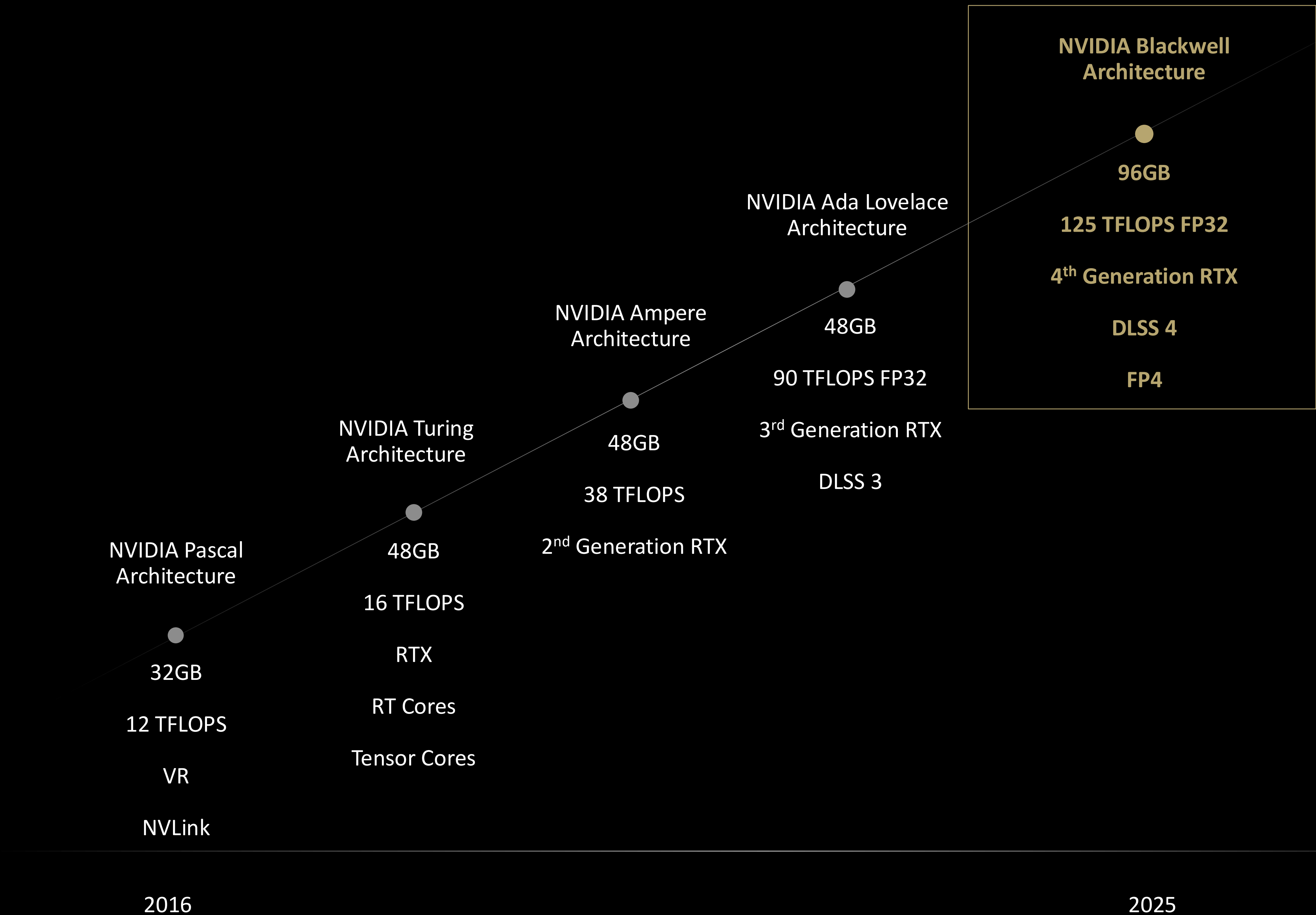
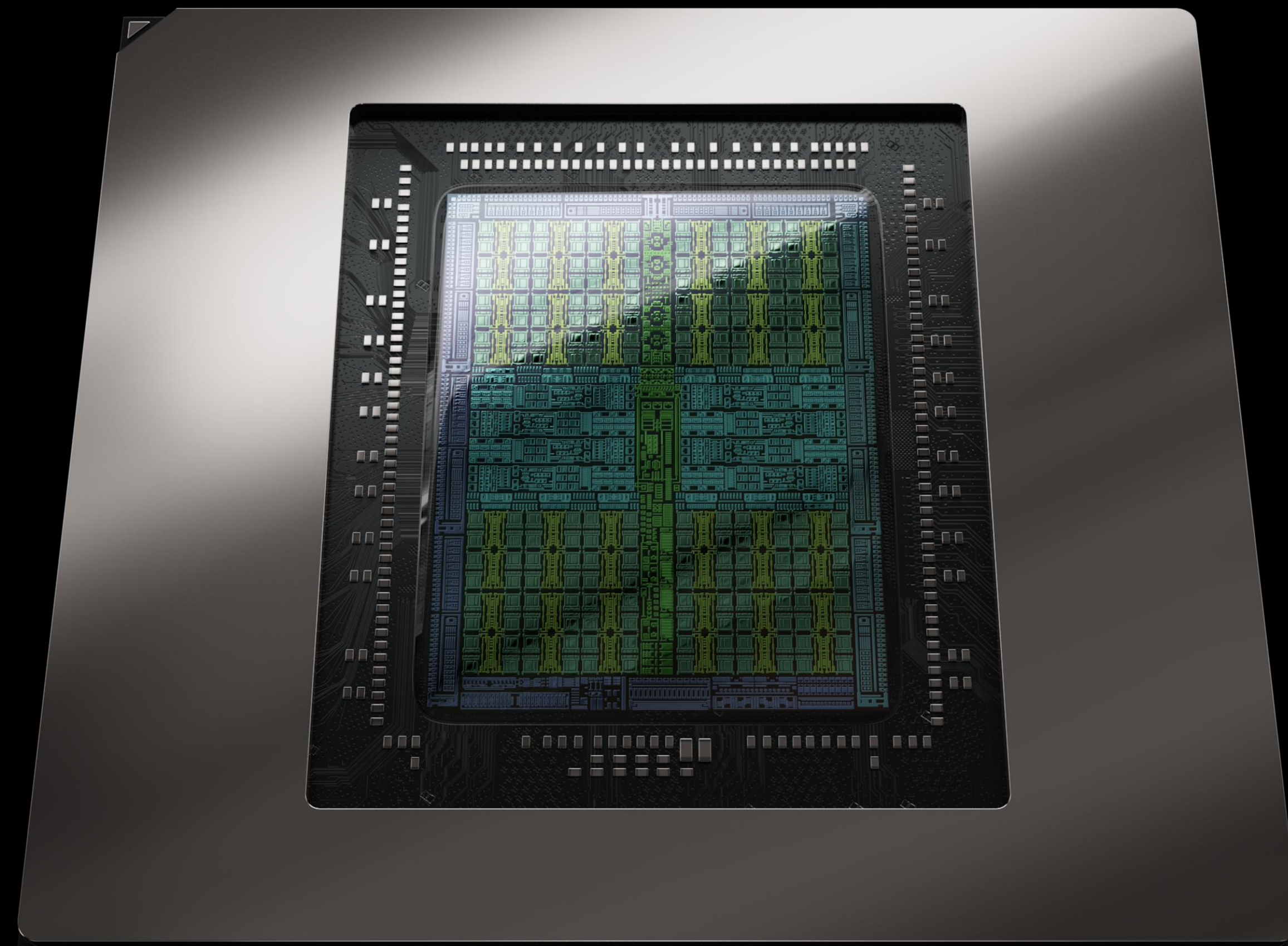




The Evolution of Hardware

Generational Leaps in GPU Technology

Continuous advancement in professional platforms & solutions



NVIDIA Blackwell Architecture

Engine for Professional AI Workflows

5th Gen Tensor Cores

4000 AI TOPS | DLSS 4, FP4

4th Gen RT Cores

380 RT TFLOPS | Mega Geometry

New Streaming Multiprocessor

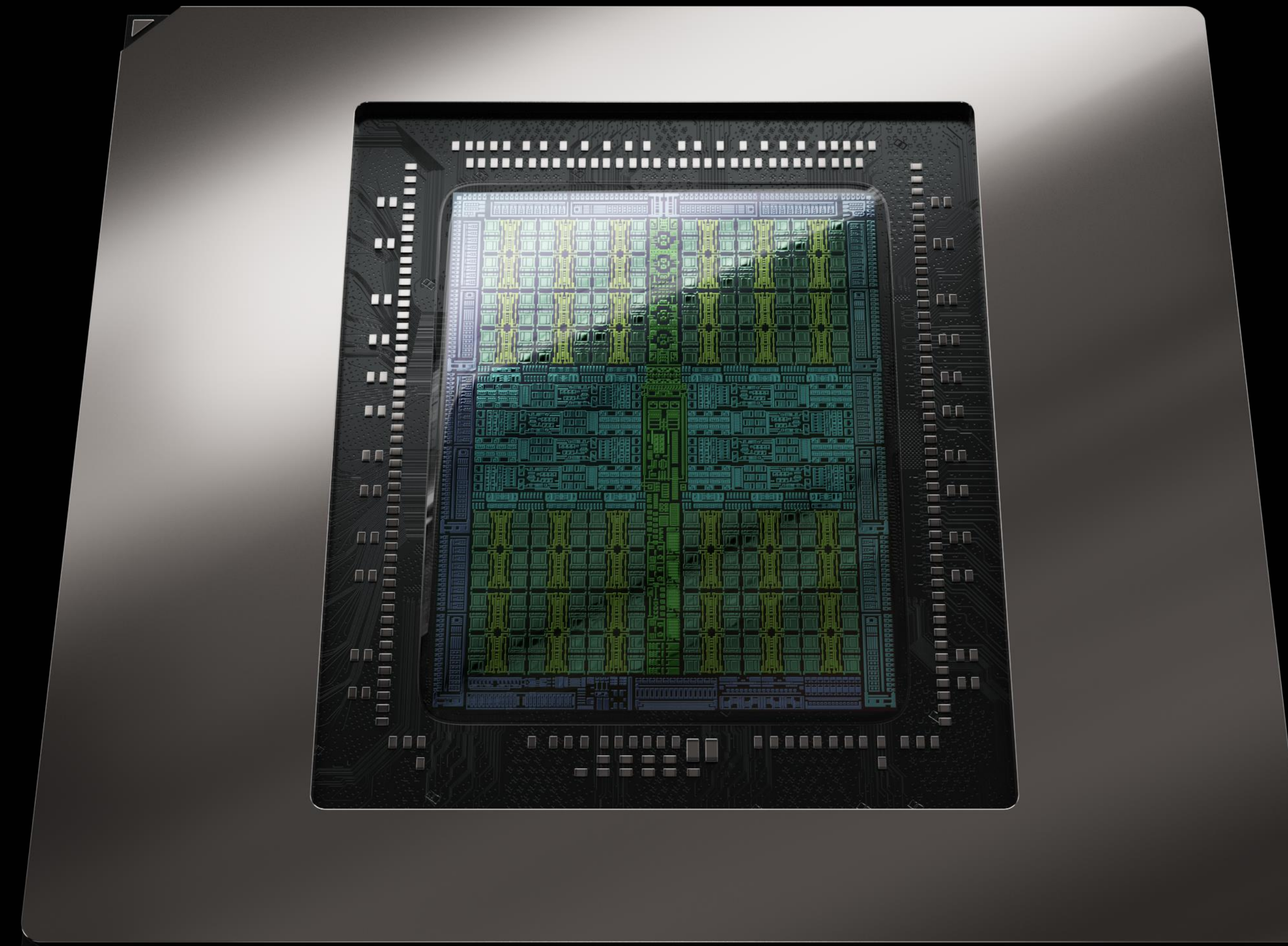
125 TFLOPS | Neural Shaders

GDDR7 Memory

Up to 96GB | 2X memory bandwidth

9th Gen NVENC / 6th Gen NVDEC

4X NVENC, 4X NVDEC with 4:2:2 support



AI Management Processor

Simultaneous AI Models + Graphics

Blackwell Max-Q

Power and space-optimized for multi-GPU

Universal MIG

Up to 4X instances

PCIe Gen 5

2X Ada

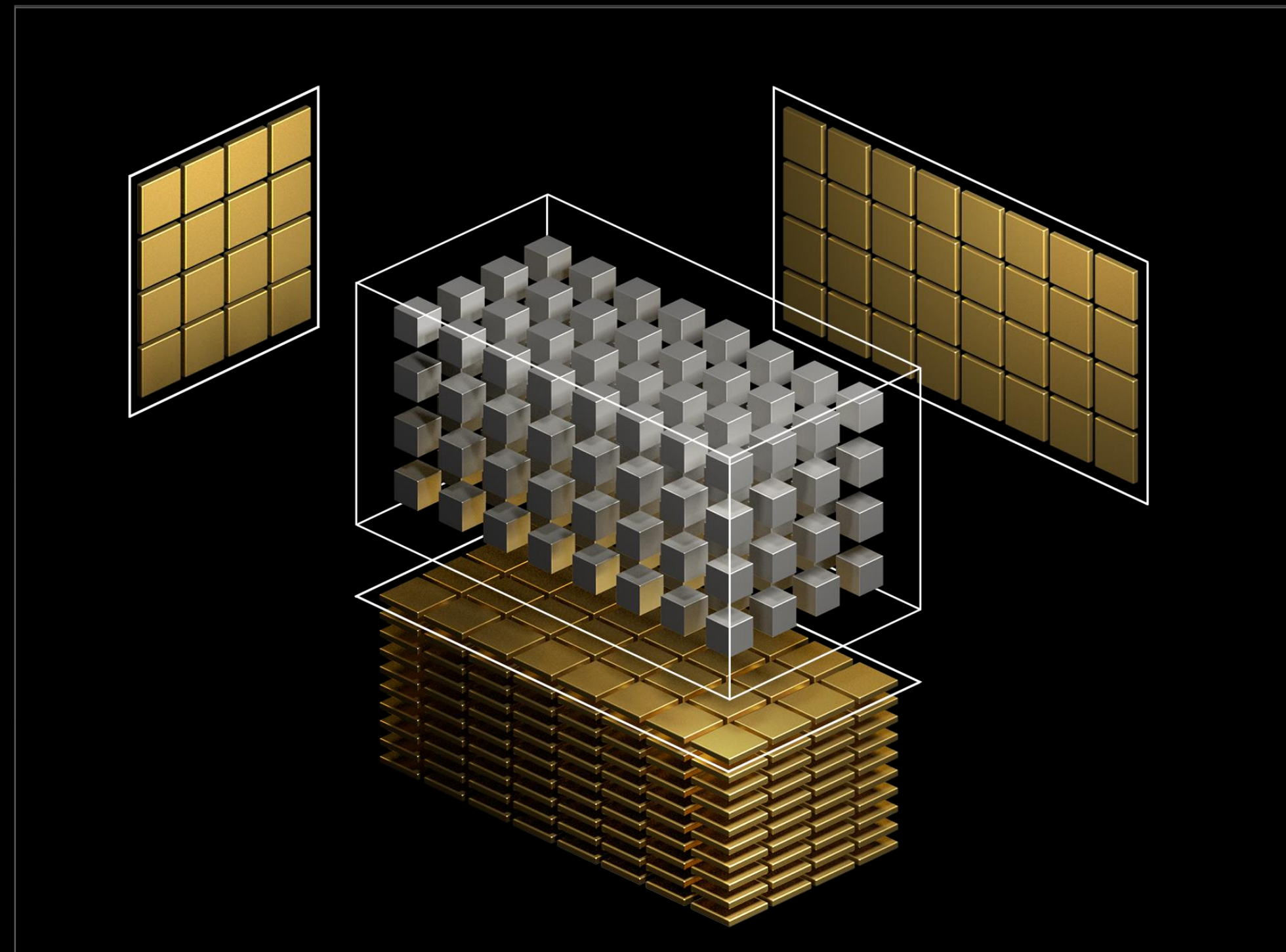
DisplayPort 2.1

8K at 165Hz

5th Generation Tensor Cores

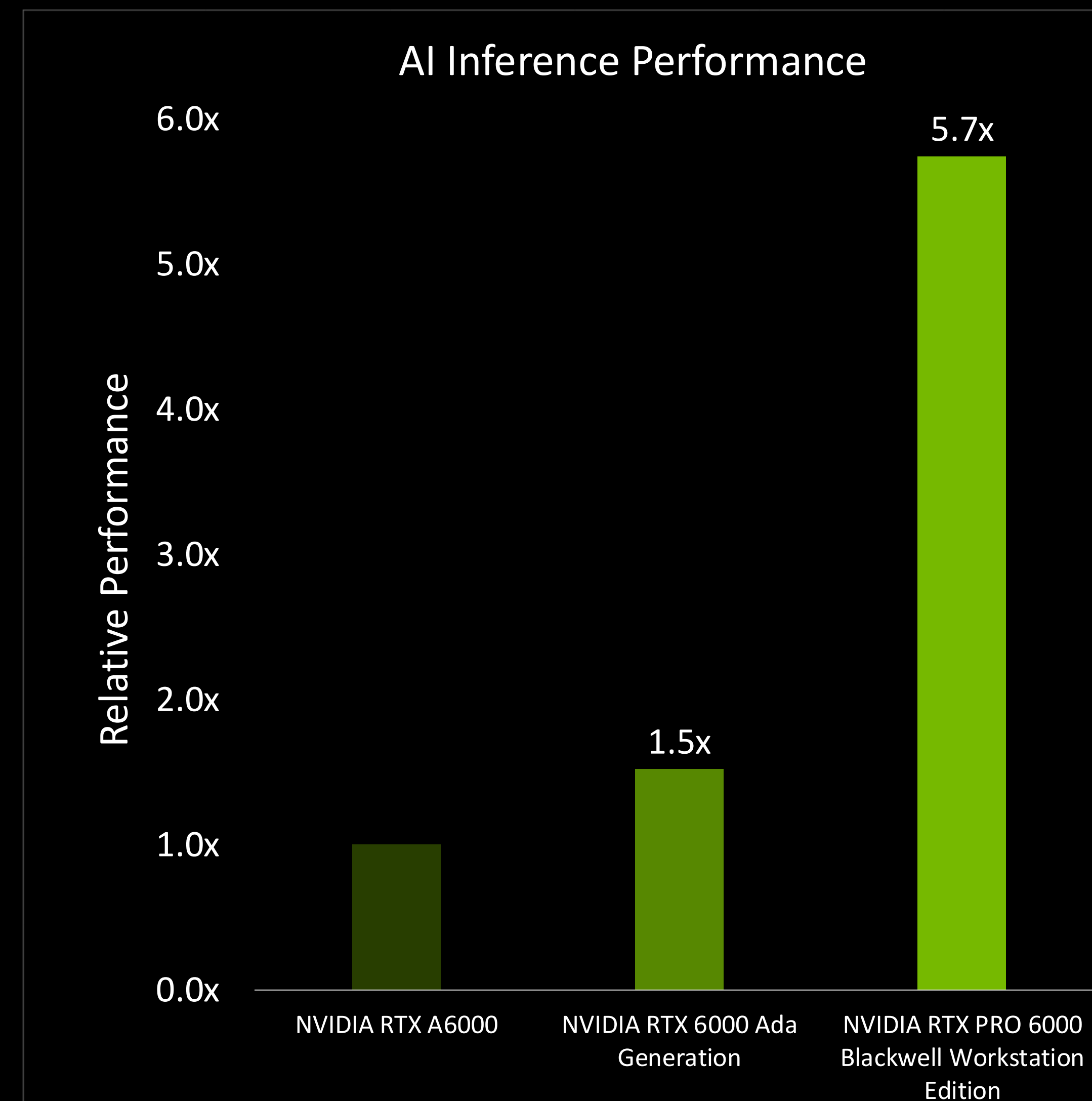
Complex AI tasks are handled with unprecedented speed and precision

2nd Gen Transformer Engine

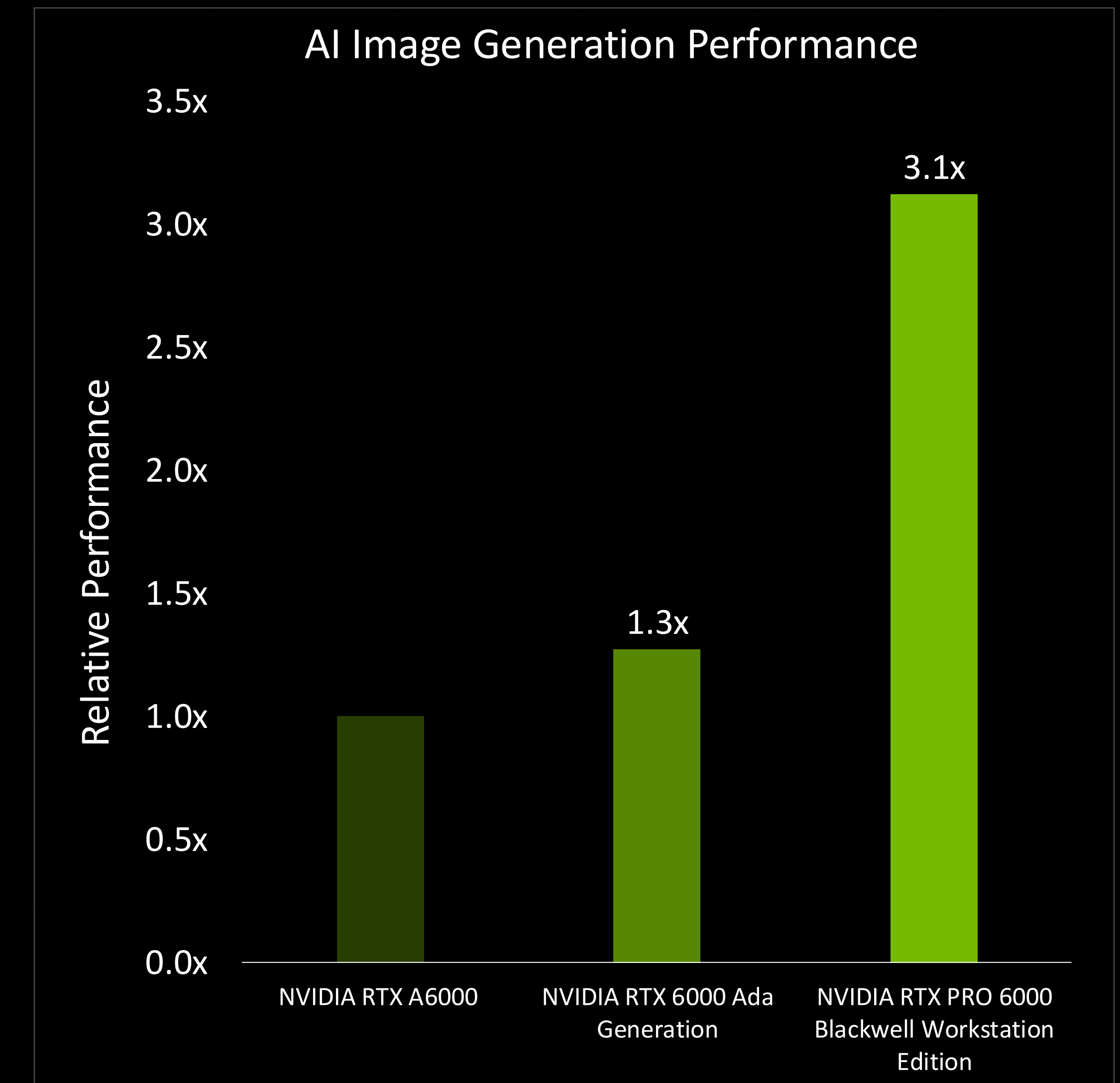


- Half the storage: FP4 uses 4 bits per number, cutting memory usage in half
- Bigger models: with FP4, fit 2x as many parameters in the same memory
- Faster AI: accelerates FP4 math, making MoE/LLM inference 2x faster than FP8
- 4000 TOPS of AI performance*

6X Faster Than Previous Generation



Over 3X Performance of Previous Generation



*Using FP4 data format with sparsity

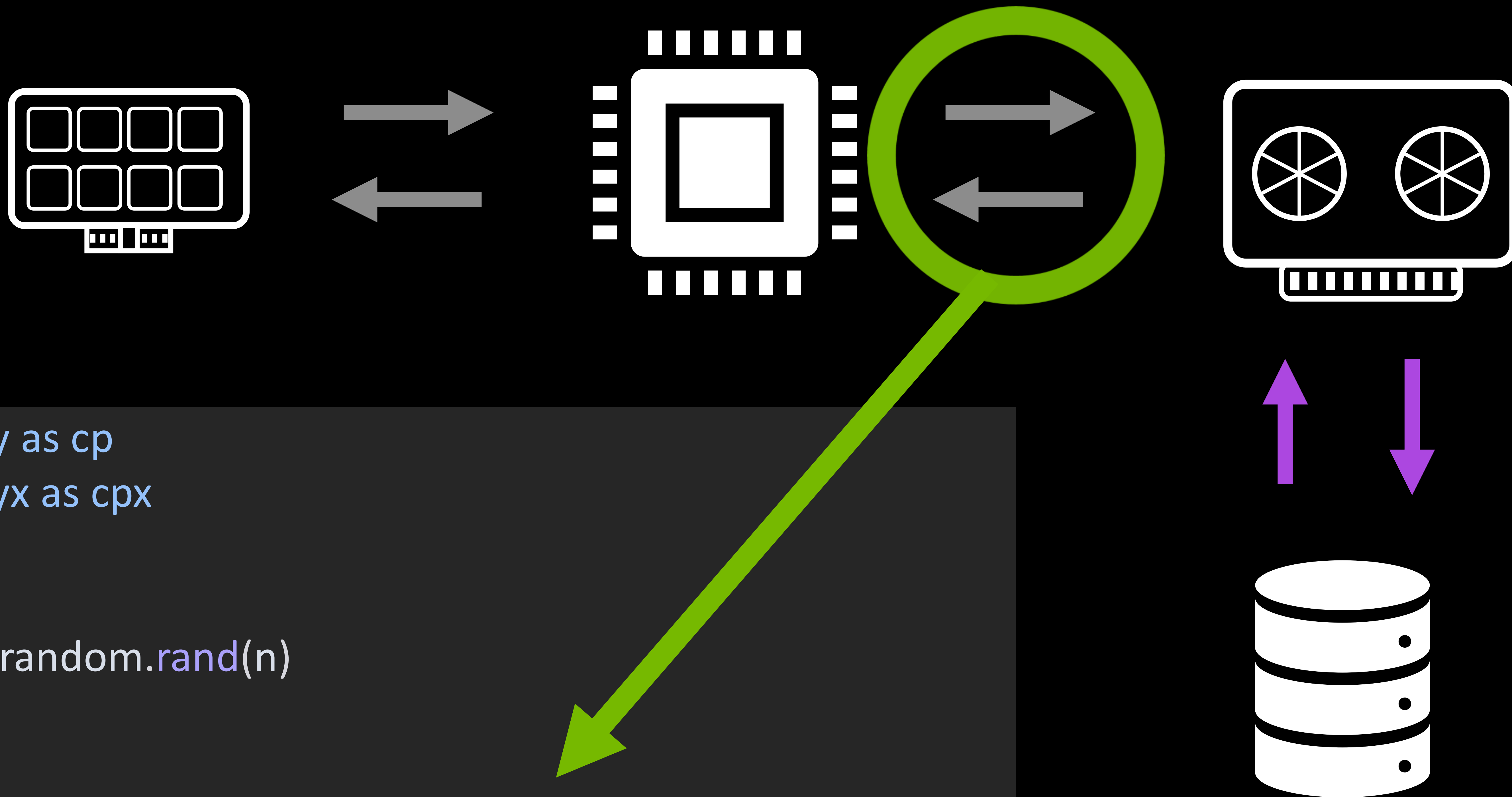
Hopper vs Blackwell Architecture

A look at the key differences

Feature	NVIDIA Hopper Architecture	NVIDIA Blackwell Architecture
Transistor Count	80 billion transistors	208 billion transistors
Manufacturing Process	TSMC 4N process	Custom-built TSMC 4NP process
Transformer Engine	First-generation	Second-generation
Decompression Engine	No	Yes
Energy Efficiency	Improved over the previous generation	25x more energy-efficient than Hopper
Interconnect Technology	Fourth-generation NVLink	Fifth-generation NVLink
Chip-to-Chip Interconnect	900 GB/s	10 TB/s
Applications	Generative AI, LLMs, Data processing, Quantum computing	Accelerated computing, AI, LLM

Moving the memory model

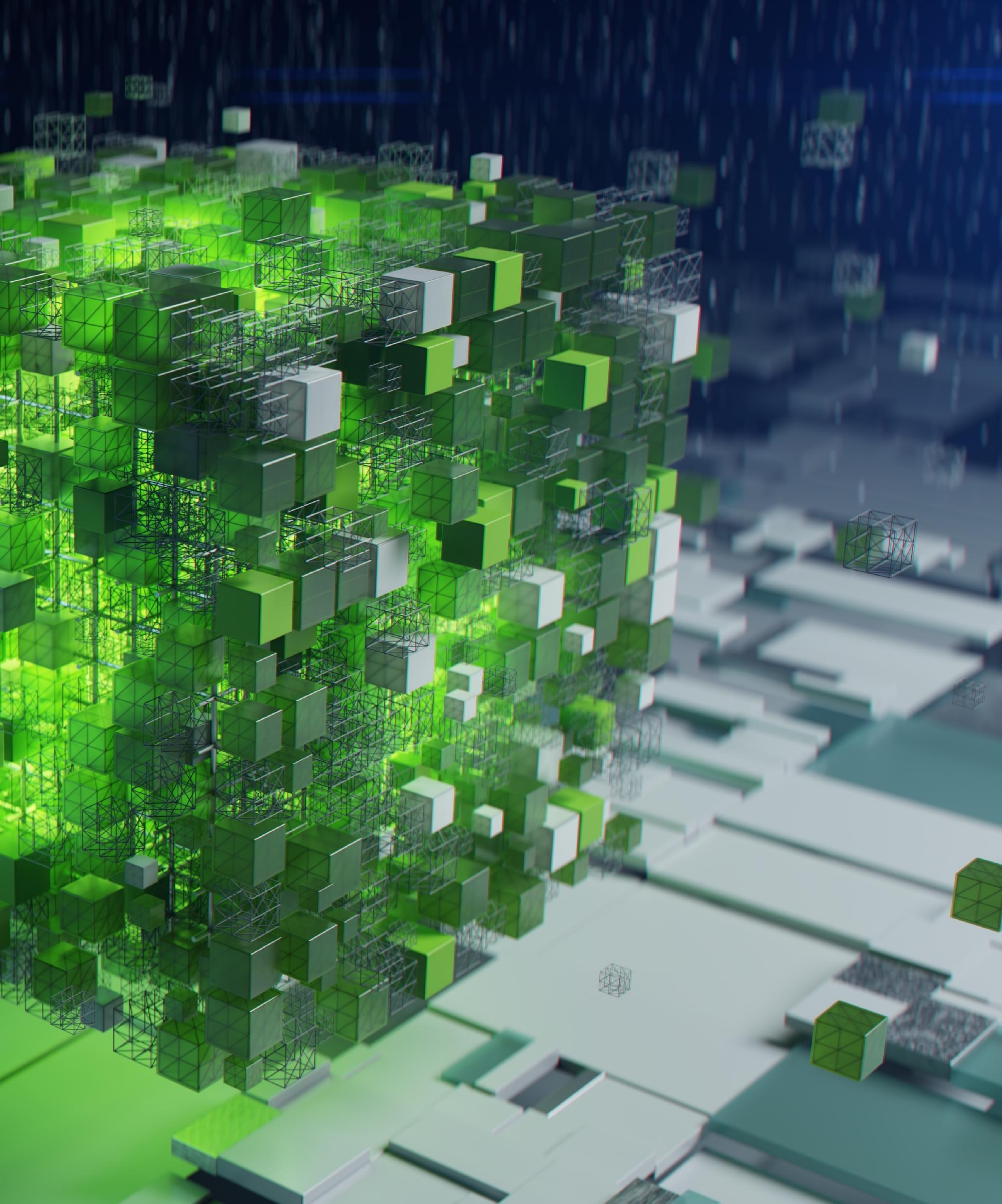
Great programmers worry about the memory



```
import cupy as cp  
import cupyx as cpx
```

```
x_cpu = np.random.rand(n)
```

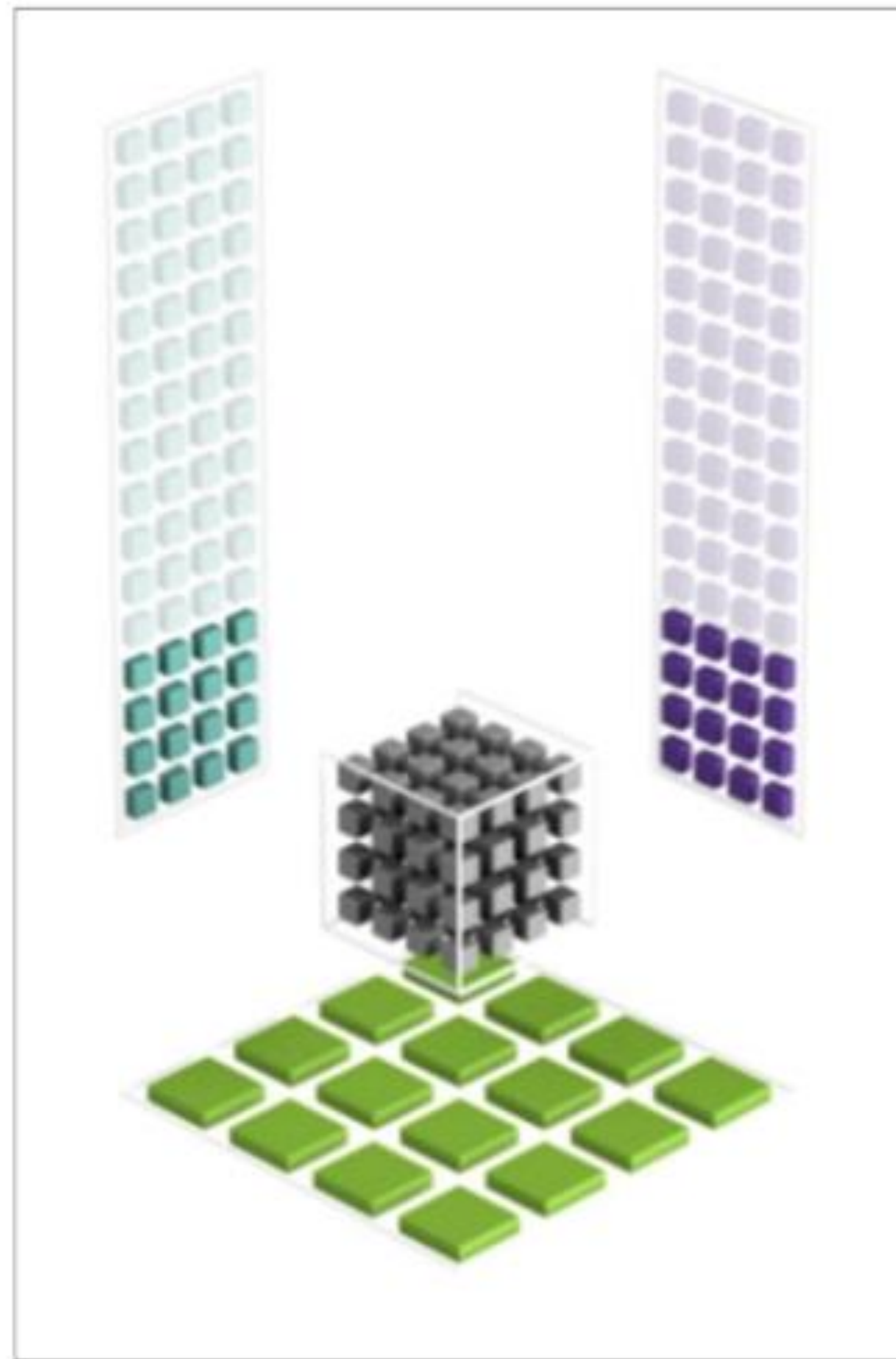
```
x_gpu = cpx.empty_like_pinned.asarray(x_cpu)
```

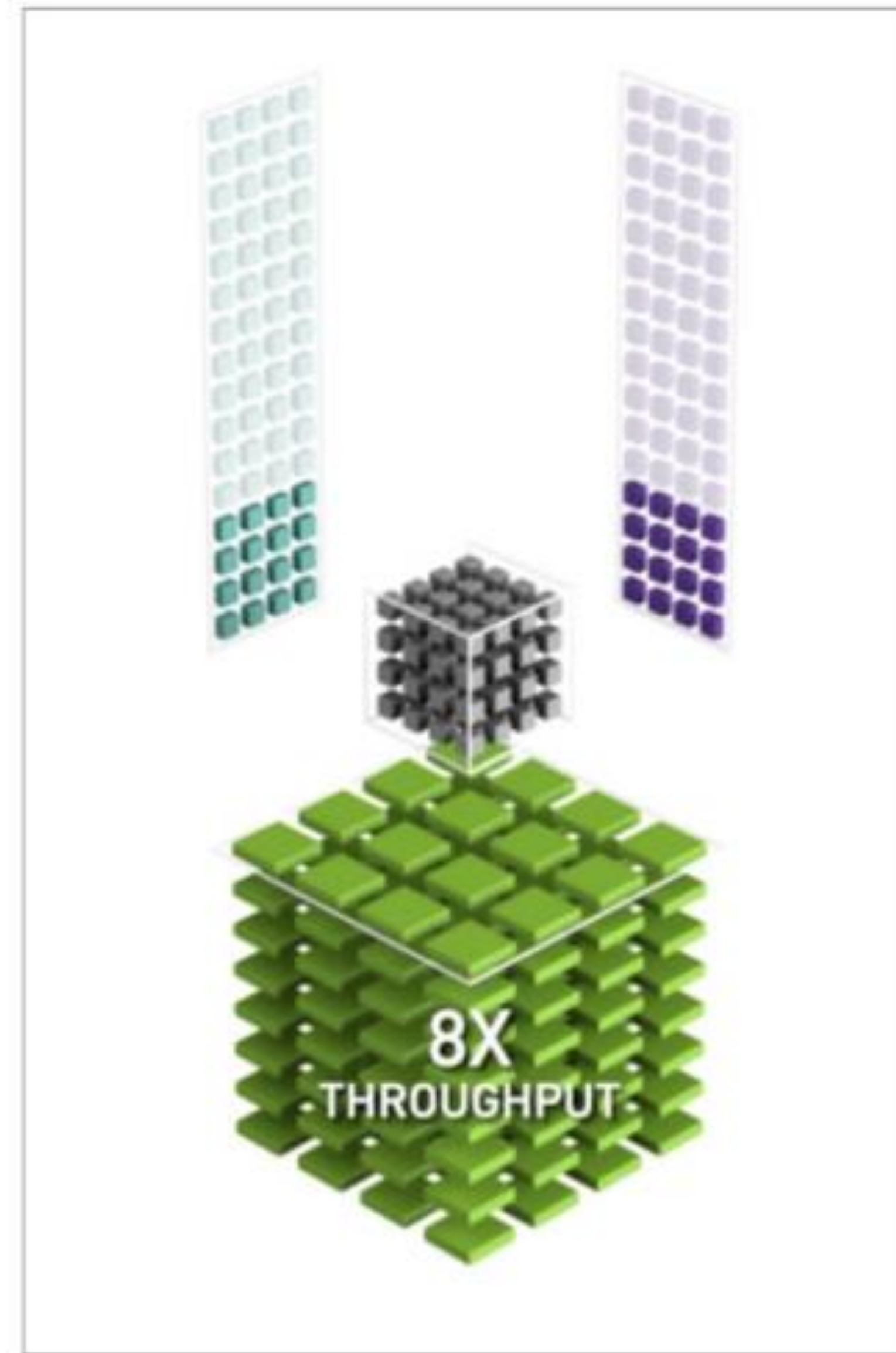
Tiling languages

The Journey to the Tensor Core

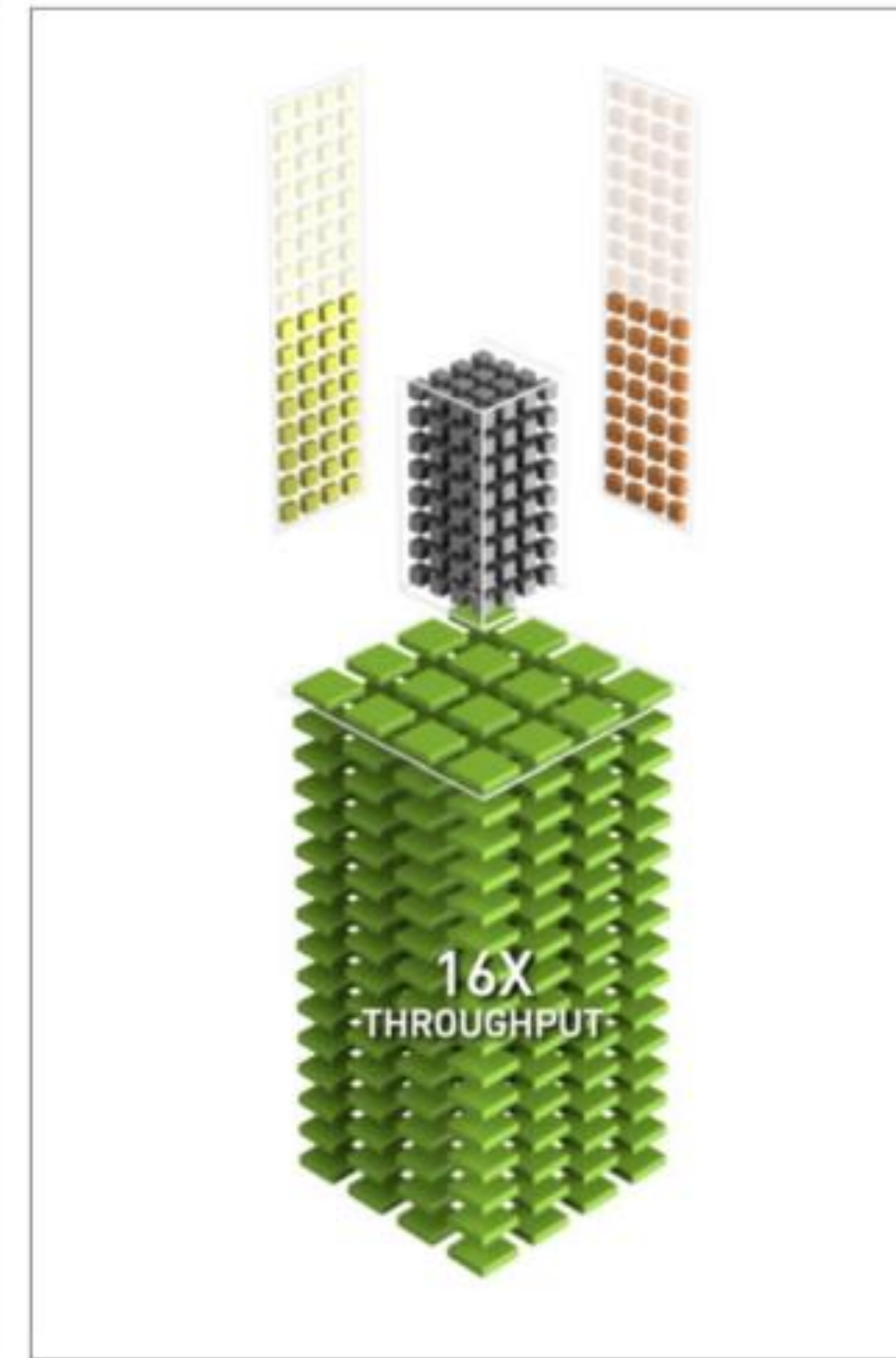
From Thread based Programming To Array based Programming



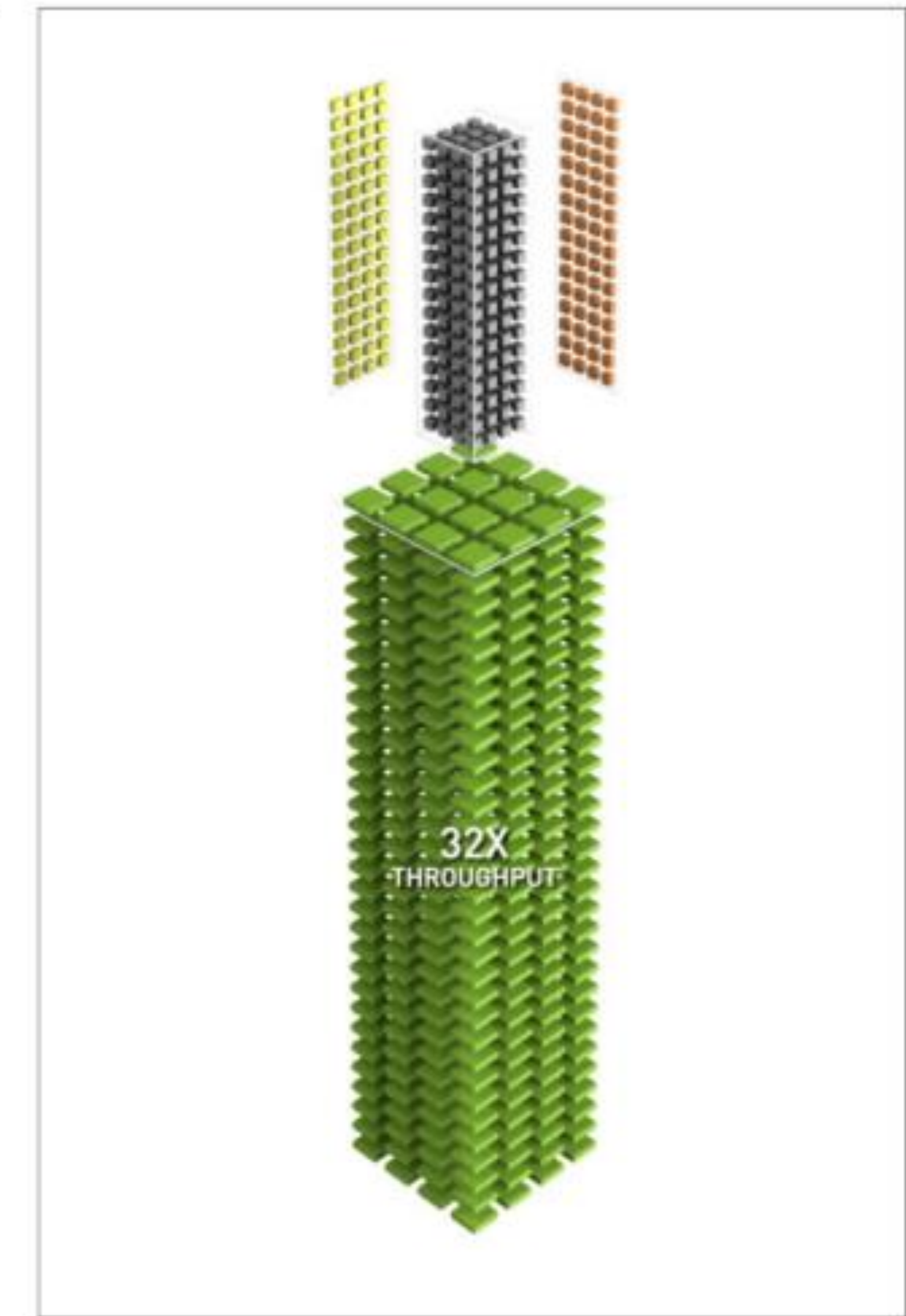
Pascal



Turing Tensor Core
FP16



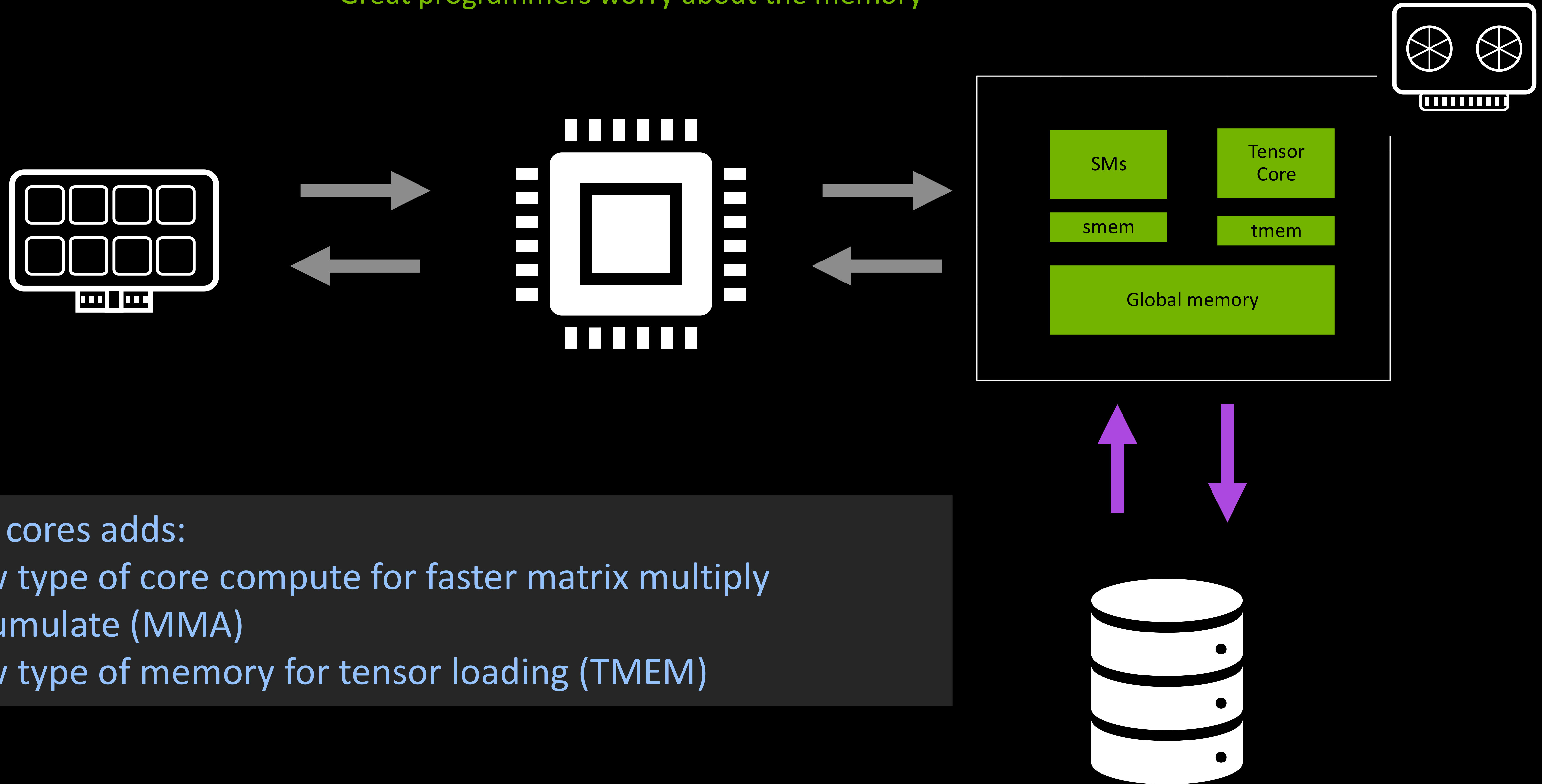
Ada Tensor Core
FP8



Blackwell Tensor Core
FP4

Moving the memory model

Great programmers worry about the memory



Tensor cores adds:

- New type of core compute for faster matrix multiply accumulate (MMA)
- New type of memory for tensor loading (TMEM)

cuTile is Pythonic CUDA

Array/tensor-based programming is more natural in Python

- **Tile Programming is designed to be intuitive to Python users**
- Array-based models are common in Python (e.g. NumPy)
- Programs have a single logical thread of control per block
- Operations are cooperative and executed in parallel by all threads in a Tileblock
- Simplifies GPU programming by handling thread management for you

NumPy Softmax

```
import numpy as np

def softmax(In, Out):

    max = np.max(In, axis=0, keepdims=True)
    num = np.exp(In - max)
    den = np.sum(num, axis=0, keepdims=True)
    smax = num / den

    Out[:] = smax
```

cuTile Softmax

```
import cuda.tile as ct

@ct.func
def softmax(In, Out, r):
    tile = ct.load(In, (0, ct.pid[0]), (In.shape[0], r))

    max = ct.max(tile, axis=0, keepdims=True)
    num = ct.exp(tile - max)
    den = ct.sum(num, axis=0, keepdims=True)
    smax = num / den

    ct.store(Out, (0, ct.pid[0]), smax)
```

Tileblocks load and compute
their own Tiles in parallel with
each other

Tile Programming is Simple and Powerful for Data-Parallel Programs

Comparison of matrix multiplication algorithm implementations

Numba SIMT Matrix Multiply

```
@cuda.jit
def fast_matmul(A, B, C):
    """
    Perform matrix multiplication of C = A * B using CUDA shared memory.

    Reference: https://stackoverflow.com/a/64198479/13697228 by @RobertCrovella
    """
    # Define an array in the shared memory
    # The size and type of the arrays must be known at compile time
    sA = cuda.shared.array(shape=(TPB, TPB), dtype=float32)
    sB = cuda.shared.array(shape=(TPB, TPB), dtype=float32)

    x, y = cuda.grid(2)

    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y
    bpg = cuda.gridDim.x    # blocks per grid

    # Each thread computes one element in the result matrix.
    # The dot product is chunked into dot products of TPB-long vectors.
    tmp = float32(0.)
    for i in range(bpg):
        # Preload data into shared memory
        sA[ty, tx] = 0
        sB[ty, tx] = 0
        if y < A.shape[0] and (tx + i * TPB) < A.shape[1]:
            sA[ty, tx] = A[y, tx + i * TPB]
        if x < B.shape[1] and (ty + i * TPB) < B.shape[0]:
            sB[ty, tx] = B[ty + i * TPB, x]

        # Wait until all threads finish preloading
        cuda.syncthreads()

        # Computes partial product on the shared memory
        for j in range(TPB):
            tmp += sA[ty, j] * sB[j, tx]

        # Wait until all threads finish computing
        cuda.syncthreads()

    if y < C.shape[0] and x < C.shape[1]:
        C[y, x] = tmp
```

cuTile-python Matrix Multiply

```
import cuda.tile as ct

@ct.func
def matmul(A: ct.Array, B: ct.Array, C: ct.Array,
           tshp: ct.Constant[ct.Shape]):

    sum = ct.zeros(tshp[0:1], A.dtype)

    pA = ct.partition(A, (tshp[0], tshp[2]))
    pB = ct.partition(B, (tshp[2], tshp[1]))

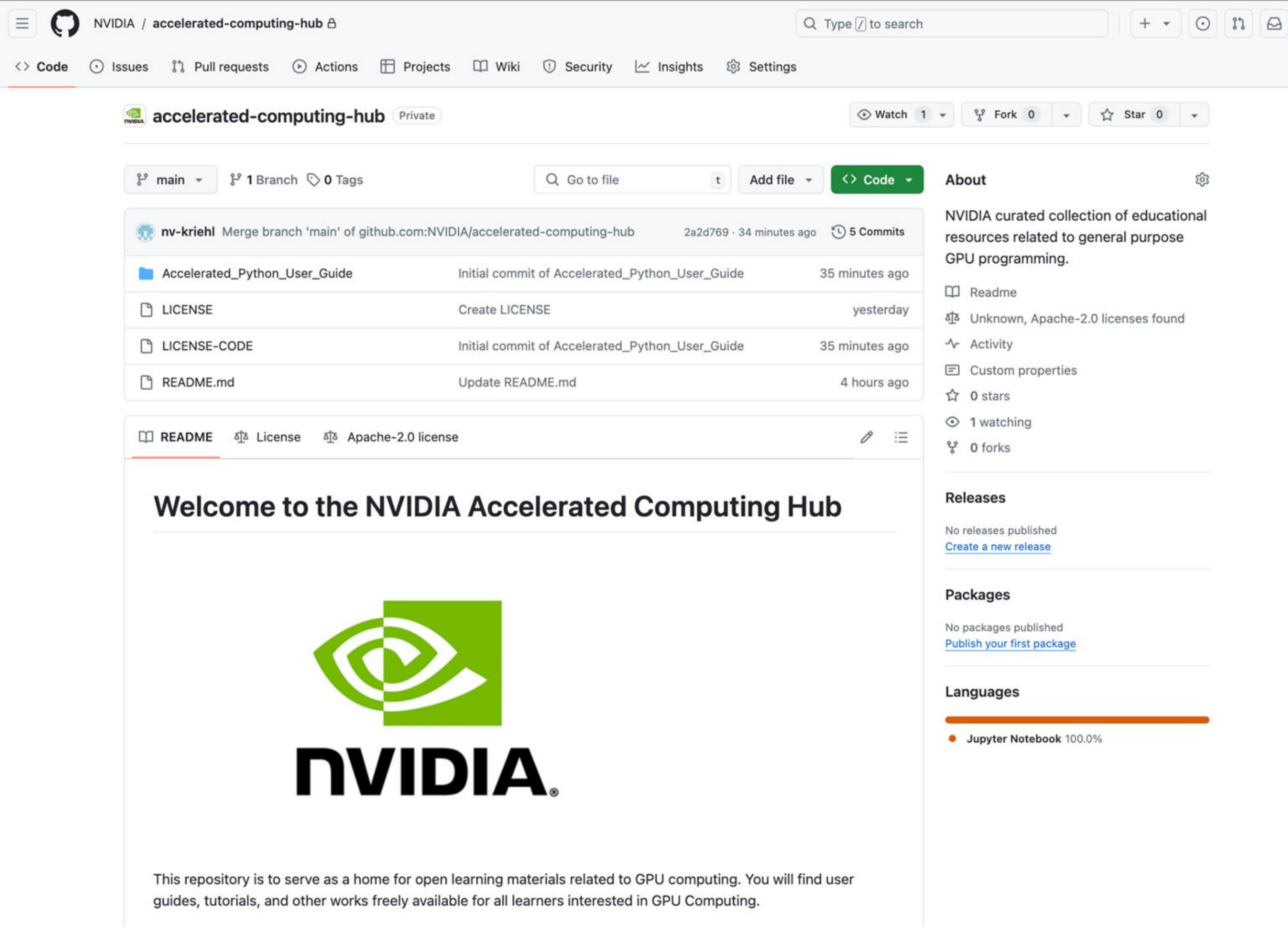
    for k in range(pA.shape[1]):
        sum = ct.mma(pA[ct.pid[0], k], pB[k, ct.pid[1]],
                    sum)

    ct.store(C, ct.pid[0:2], sum)
```


Accelerated Computing Hub

Open source resource for learning CUDA

Clone it, Run it, Modify it, Share it



<https://github.com/NVIDIA/accelerated-computing-hub>

Grow Your Skills with NVIDIA Learning Paths

Build expertise in AI, accelerated computing, and graphics through guided NVIDIA learning paths

High-Level Benefits

- Follow structured, role-based skill development
- Learn through hands-on labs and real-world projects
- Earn industry-recognized certifications



<https://nvda.ws/3ZXQYb7>

